

RAPPORT DE PROJET DE FIN D'ETUDES

[Nom de l'etablissement -- a completer]

CHAPITRE 3

ANALYSE ET CONCEPTION DU SYSTEME

Plateforme de gestion industrielle et commerciale PROBAR

Realise par : Hadil Ibn Haj Fraj

[Encadrant academique -- a completer]

[Encadrant professionnel -- a completer]

Annee universitaire 2025 - 2026

SOMMAIRE

3.1 Introduction	4
3.2 Contexte metier	5
3.3 Identification des acteurs	7
3.4 Analyse des besoins fonctionnels	9
3.5 Diagramme global des cas d'utilisation	11
3.6 Description detaillee des cas d'utilisation	14
3.7 Diagrammes d'activite	21
3.8 Diagramme de classes	30
3.9 Diagrammes de sequence	32
3.10 Architecture fonctionnelle du systeme	37
3.11 Synthese de la conception	39
3.12 Conclusion	40
* Points a verifier dans le projet	41

Liste des figures

Figure 3.1 — Diagramme de contexte metier	6
Figure 3.2 — Diagramme global des cas d'utilisation	12
Figure 3.3 — Diagramme d'activite : Authentification	22
Figure 3.4 — Diagramme d'activite : Creation d'une fiche de production (PROBAR / MAINTENANCE)	23
Figure 3.5 — Diagramme d'activite : Creation d'une fiche MELANGE	24
Figure 3.6 — Diagramme d'activite : Consultation de l'historique	25
Figure 3.7 — Diagramme d'activite : Modification d'une fiche	26
Figure 3.8 — Diagramme d'activite : Suppression d'une fiche	27
Figure 3.9 — Diagramme d'activite : Upload d'un document	28
Figure 3.10 — Diagramme d'activite : Traitement d'une demande de maintenance	29
Figure 3.11 — Diagramme de classes global	30
Figure 3.12 — Diagramme de classes du module Production	30
Figure 3.13 — Diagramme de classes du module CRM	30
Figure 3.14 — Diagramme de classes du module Demandes internes	31
Figure 3.15 — Diagramme de sequence : Authentification	32
Figure 3.16 — Diagramme de sequence : Creation d'une fiche MELANGE	33
Figure 3.17 — Diagramme de sequence : Consultation de l'historique	33
Figure 3.18 — Diagramme de sequence : Modification d'une fiche	34
Figure 3.19 — Diagramme de sequence : Suppression d'une fiche	35
Figure 3.20 — Diagramme de sequence : Upload d'un document	35
Figure 3.21 — Architecture fonctionnelle en couches	37

Liste des tableaux

Tableau 3.1 — Identification des acteurs	7
Tableau 3.2 — Synthèse des besoins fonctionnels	9
Tableau 3.3 — Besoins non fonctionnels identifiés	10
Tableau 3.4 — Classes principales du modèle métier	31
Tableau 3.5 — Rôle de chaque couche de l'architecture	37
Tableau 3.6 -- Points à vérifier dans le projet	41

3.1 Introduction

Ce chapitre a pour objectif de transformer les besoins metier identifies au sein de la plateforme PROBAR en une conception logicielle structuree, conforme au formalisme UML (Unified Modeling Language). Il constitue la charniere entre l'analyse fonctionnelle du systeme et sa realisation technique, presentee dans le chapitre suivant.

La demarche d'analyse suivie repose sur trois etapes complementaires. La premiere consiste a identifier les acteurs reels du systeme, c'est-a-dire les profils d'utilisateurs effectivement geres par la plateforme, tels qu'ils sont definis dans le modele de donnees et controles par les mecanismes d'autorisation du backend. La deuxieme etape porte sur l'analyse des besoins fonctionnels, recensee module par module a partir des routes de l'API REST, des ecrans de l'application Flutter et des services associes. La troisieme etape traduit ces besoins en diagrammes UML : cas d'utilisation, activites, classes et sequences, avant de presenter l'architecture fonctionnelle globale du systeme.

Contrairement a une demarche theorique fondee sur un cahier des charges abstrait, l'analyse presentee ici s'appuie exclusivement sur l'examen direct du code source du projet : le backend Node.js/Express (architecture modulaire avec ORM Sequelize sur base de donnees PostgreSQL) et le frontend Flutter Web. Chaque acteur, chaque fonctionnalite, chaque classe et chaque route API presentes dans ce chapitre correspond a un element effectivement implemente et verifiable dans le code source. Lorsqu'une information n'a pas pu etre confirmee avec certitude, elle est explicitement signalee par la mention « A verifier », conformement a la rigueur attendue dans un travail d'ingenierie logicielle.

Un point methodologique important doit etre souligne d'emblee : le nom commercial PROBAR designe a la fois la societe cliente pour laquelle la plateforme a ete developpee (le domaine de messagerie « probardistribution.com » intervient dans l'attribution automatique des roles) et l'une des deux familles de production gerees par le systeme, l'autre etant PROMESH. La plateforme reelle n'est donc pas un progiciel de gestion generique organise autour de modules Finance/Facturation/Expedition, mais une plateforme hybride combinant un CRM (gestion de la relation client) et un systeme de suivi de production industrielle, complete par des workflows de demandes internes et un module de copilote conversationnel fonde sur l'intelligence artificielle.

Ce chapitre se conclut par une synthese de la conception (section 3.11) qui met en relation les choix de modelisation avec l'architecture retenue, puis par une section « Points a verifier » qui recense de maniere transparente les zones d'incertitude releves lors de l'analyse.

3.2 Contexte metier

La plateforme PROBAR a ete developpee pour le compte de la societe Probar Distribution (integrateur : CBI Tunisia) afin de centraliser deux familles de besoins auparavant geres de maniere dispersee au sein de l'entreprise.

3.2.1 Activite de l'entreprise et problematiques traitees

La premiere famille de besoins concerne le suivi de la production industrielle de deux familles de produits reelles identifiees dans le code, PROBAR et PROMESH, incluant les operations de melange de matieres, les controles qualite, le suivi des arrêts machine, la declaration des non-conformites et le suivi des matieres recuperables (chutes et dechets de production). La seconde famille de besoins concerne la gestion commerciale et de projet (CRM) : suivi des projets clients, gestion des contacts commerciaux et de leur relance, pilotage d'un pipeline de vente, depot de devis et de bons de commande, ainsi que la dematerialisation de trois types de demandes internes : demandes de maintenance des equipements, demandes de ressources humaines (conges, sorties) et demandes d'archivage de dossiers projet.

La plateforme repond ainsi a des problematiques concretes de tracabilite (chaque fiche de production est horodatee et rattachee a l'utilisateur qui l'a saisie), de fiabilisation des processus qualite (les controles et non-conformites sont formalises et verrouillables apres validation), de suivi commercial structure (pipeline de vente et relances planifiees) et de dematerialisation des demandes internes (initialement traitees, selon toute vraisemblance, sur support papier ou par des canaux non centralises — information non confirmee par le code, « a verifier » aupres de l'equipe projet).

3.2.2 Utilisateurs et modules

Les utilisateurs de la plateforme se repartissent en plusieurs profils fonctionnels detailles en section 3.3 : personnel d'encadrement (administrateurs), responsable logistique et production, personnel commercial, personnel d'accueil, et utilisateurs standards soumettant des demandes internes. Les modules metier identifies dans le code sont presentes en detail en section 3.4.

3.2.3 Flux d'information et interactions

Sur le plan technique, le frontend Flutter Web communique avec le backend Express exclusivement via une API REST securisee par jeton JWT (JSON Web Token), dans lequel le role de l'utilisateur est directement embarque. Les donnees sont persistees dans une base PostgreSQL par l'intermediaire de l'ORM Sequelize. Un canal de communication temps reel (Socket.IO) complete l'API REST pour la diffusion immediate des notifications aux utilisateurs connectes. Les documents (pieces jointes, devis, bons de commande, photos de non-conformite) sont stockes sur le systeme de fichiers du serveur et exposes en lecture via une route statique dediee.

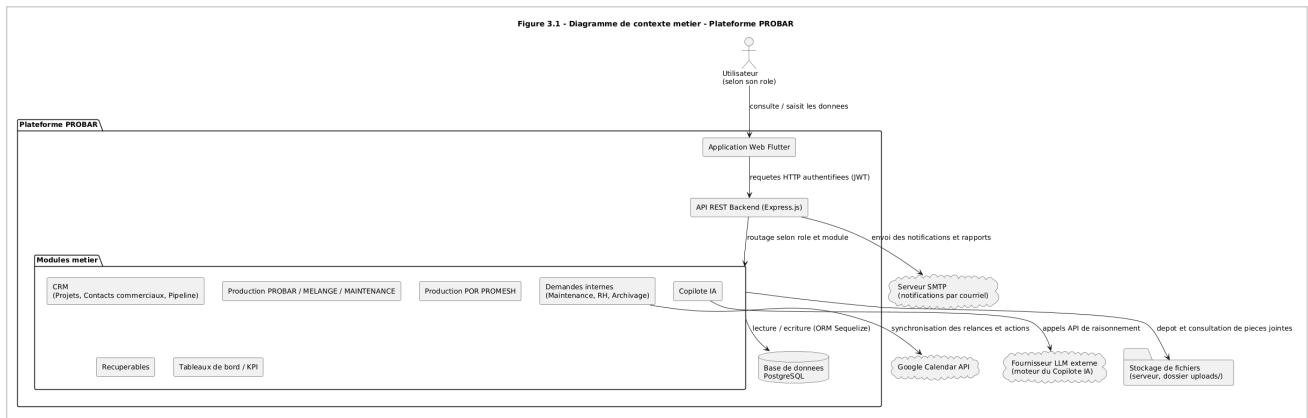


Figure 3.1 — Diagramme de contexte metier

L'utilisateur interagit uniquement avec l'application Flutter, qui ne dialogue jamais directement avec la base de données : toute operation transite par l'API REST, laquelle orchestre les modules metier avant d'accéder a PostgreSQL par l'intermediaire de l'ORM. Trois systemes externes reels ont ete identifiés dans le code source : l'API Google Calendar (synchronisation bidirectionnelle des relances commerciales et des actions projet), un serveur SMTP (envoi des courriels de notification, de reinitialisation de mot de passe et des rapports) et un fournisseur de modele de langage externe consomme par le module Copilote IA. Aucun service d'OCR externe n'a ete trouve dans le code (voir section « Points a verifier »).

3.3 Identification des acteurs

L'analyse du modele `User` (`src/models/User.js`) revele une enumeration de role stricte, prise en charge par PostgreSQL : `user`, `commercial`, `accueil`, `admin`, `superadmin`, `superadmin2`, `responsable\_logistique\_achat`. L'examen croise des middlewares d'autorisation du backend (`moduleAccessGuard.js`, `requireRole.js`, `kpiRoleGuard.js`, `requireHrReviewer.js`, `requireMaintenanceManager.js`, `requireRootAdmin.js`) permet d'etablir les responsabilites reelles associees a chaque role.

Tableau 3.1 — Identification des acteurs

Acteur	Description	Responsabilites	Modules accessibles	Permissions
Administrateur	Compte de gestion courante de la plateforme (role `admin`).	Activation des comptes utilisateurs ; configuration du pipeline commercial et des types d'action ; supervision et suppression des fiches de production ; traitement des demandes RH et de maintenance ; consultation des indicateurs globaux.	Tous les modules (aucune restriction `moduleAccessGuard`).	READ_WRITE_ROLES et DELETE_ROLES sur les modules de production ; HR reviewer ; maintenance manager ; KPI globaux (`kpiRoleGuard`).
Super Administrateur	Extension du role Administrateur (roles `superadmin` et `superadmin2`, strictement identiques dans le code).	Toutes les responsabilites de l'Administrateur, augmentees du droit exclusif de modifier le role d'un autre utilisateur.	Tous les modules.	Seul profil habilite a executer `PUT /admin/users/:id/role`.
Responsable Logistique & Achat	Role operationnel de terrain (role `responsable_logistique_achat`), restreint par le middleware `moduleAccessGuard`.	Saisie et suivi des fiches PROBAR / MELANGE / MAINTENANCE et POR PROMESH ; gestion des fiches Recuperables ; soumission de demandes RH ; soumission et traitement des demandes de maintenance ; usage du Copilote IA.	Uniquement : /por-promesh, /industrial-records, /hr-requests, /recuperables, /maintenance-requests, /auth, /users/me, /uploads, /ai-copilot.	Ni suppression de fiche (reservee aux roles administrateurs), ni acces au CRM, ni acces aux tableaux de bord globaux.
Commercial	Role dedie a la prospection (role `commercial`), attribue automatiquement a toute adresse se terminant par « @probardistribution.com ».	Gestion des contacts commerciaux, des actions et relances associees ; consultation du tableau de bord commercial ; suivi des projets clients.	CRM (projets, contacts commerciaux) ; tableau de bord commercial.	Aucune restriction `moduleAccessGuard` specifique au-dela de l'authentification generale.
Accueil	Role attribue automatiquement a une adresse e-mail precise (role `accueil`).	Perimetre fonctionnel precis non confirme dans le code au-dela de son exclusion de la liste des commerciaux eligibles aux relances (`NON_COMMERCIAL_ROLE`).	A verifier dans le projet.	A verifier dans le projet.

Acteur	Description	Responsabilites	Modules accessibles	Permissions
Utilisateur	Role attribue par default a tout nouveau compte (role `user`).	Gestion des projets dont il est proprietaire ou membre ; soumission et suivi de ses propres demandes de maintenance, RH et d'archivage ; consultation de son tableau de bord personnel.	Modules CRM (selon appartenance au projet) ; demandes internes (en tant que demandeur) ; tableau de bord personnel.	Pas d'accès aux fiches de production (PROBAR/MELANGE/MAINTENANCE/POR PROMESH), reservees a `READ_WRITE_ROLES`.

Conformement au principe de representation UML retenu, les roles `superadmin` et `superadmin2` — strictement identiques dans toutes les regles d'autorisation relevees dans le code — sont representes par un unique acteur « Super Administrateur ». Une relation de generalisation (Super Administrateur herite d'Administrateur) est utilisee dans les diagrammes de cas d'utilisation, ce role possedant l'integralite des droits de l'Administrateur, augmentee du droit de gestion des roles.

Une particularite de l'implementation actuelle merite d'etre signalee : plusieurs permissions sensibles ne reposent pas exclusivement sur le role mais sur une adresse e-mail precise codee en dur dans la configuration du backend : l'approbation ou le rejet des demandes d'archivage est reserve au compte dont l'adresse correspond a `ROOT\_ADMIN\_EMAIL` (`src/config/rootAdmin.js`), la gestion RH admet egalement l'adresse definie par `HR\_MANAGER\_EMAIL` (`src/config/hrManager.js`), et la gestion de la maintenance admet l'adresse definie par `MAINTENANCE\_MANAGER\_EMAIL` (`src/config/maintenanceManager.js`), en plus des roles administrateurs. Ce mecanisme hybride role/e-mail est une caracteristique reelle et notable de la conception actuelle du systeme d'autorisation.

3.4 Analyse des besoins fonctionnels

Cette section recense l'ensemble des besoins fonctionnels identifiés module par module, à partir des routes de l'API REST (`src/routes/`, `src/modules/\*/routes/`), des contrôleurs, des services backend et des écrans Flutter correspondants. Le tableau ci-dessous constitue l'inventaire synthétique de ces besoins ; les cas d'utilisation les plus significatifs font l'objet d'une description détaillée (préconditions, scénario nominal, scénarios alternatifs, exceptions, postconditions) en section 3.6, afin d'éviter une double redondance de contenu.

Tableau 3.2 — Synthèse des besoins fonctionnels

ID	Nom du besoin	Module	Acteur principal	Objectif
BF-01	Se connecter	Authentification	Tout utilisateur	Accéder à l'application avec les droits liés à son rôle.
BF-02	Se déconnecter	Authentification	Tout utilisateur	Invalidiser la session en cours.
BF-03	Reinitialiser le mot de passe	Authentification	Tout utilisateur	Récupérer l'accès au compte en cas d'oubli.
BF-04	Créer une fiche PROBAR / MAINTENANCE	Production (Industrial Records)	Resp. Logistique & Achat	Enregistrer une fiche de production ou d'intervention.
BF-05	Créer une fiche MELANGE	Production (Industrial Records)	Resp. Logistique & Achat	Documenter un cycle de mélange de matières (assistant 3 étapes).
BF-06	Consulter une fiche PROBAR/MELANGE/ MAINTENANCE	Production (Industrial Records)	Resp. Logistique & Achat, Admin	Afficher le détail d'une fiche existante.
BF-07	Modifier une fiche PROBAR/MELANGE/ MAINTENANCE	Production (Industrial Records)	Resp. Logistique & Achat, Admin	Mettre à jour les données d'une fiche existante.
BF-08	Supprimer une fiche PROBAR/MELANGE/ MAINTENANCE	Production (Industrial Records)	Administrateur	Purger définitivement une fiche.
BF-09	Créer / compléter une fiche POR PROMESH	Production (POR PROMESH)	Resp. Logistique & Achat	Documenter un cycle de production PROMESH complet (personnel, rendement, contrôles).
BF-10	Valider (verrouiller) une fiche POR PROMESH	Production (POR PROMESH)	Resp. Logistique & Achat	Clore définitivement une fiche après contrôle.
BF-11	Déclarer une non-conformité	Production (POR PROMESH)	Resp. Logistique & Achat	Tracer un écart qualité avec pièce jointe éventuelle.
BF-12	Consulter le tableau de bord / stats PROMESH	Production (POR PROMESH)	Resp. Logistique & Achat, Admin	Visualiser les indicateurs de production PROMESH.
BF-13	Créer une fiche Recupérables	Recupérables	Resp. Logistique & Achat	Ouvrir le suivi des chutes/déchets pour une combinaison machine/ligne/poste/date.
BF-14	Cloturer une fiche Recupérables	Recupérables	Resp. Logistique & Achat	Terminer le suivi d'une fiche de récupérables.
BF-15	Créer / suivre un projet client	CRM — Projets	Utilisateur, Commercial, Admin	Piloter un dossier client de sa création à sa clôture.
BF-16	Déplacer un projet dans le pipeline	CRM — Pipeline	Utilisateur, Commercial, Admin	Faire progresser un projet dans le cycle de vente.
BF-17	Déposer un devis / bon de commande	CRM — Projets	Utilisateur, Commercial, Admin	Rattacher un document commercial à un projet.
BF-18	Gérer un contact commercial	CRM — Contacts commerciaux	Commercial	Créer et qualifier un prospect.
BF-19	Planifier une relance commerciale	CRM — Contacts commerciaux	Commercial	Programmer un rappel de suivi client, synchronisable au calendrier.
BF-20	Soumettre une demande de maintenance	Demandes internes	Utilisateur	Signaler une panne d'équipement.

ID	Nom du besoin	Module	Acteur principal	Objectif
BF-21	Traiter une demande de maintenance	Demandes internes	Resp. Logistique & Achat, Admin	Accepter, affecter, demarrer et cloturer un ticket.
BF-22	Soumettre une demande RH	Demandes internes	Utilisateur	Demander un conge ou une sortie.
BF-23	Traiter une demande RH	Demandes internes	Administrateur	Accepter, rejeter ou traiter une demande RH.
BF-24	Demander l'archivage d'un projet	Demandes internes	Utilisateur, Commercial, Admin	Initier l'archivage ou le desarchivage d'un dossier.
BF-25	Approuver l'archivage d'un projet	Demandes internes	Administrateur (compte racine)	Statuer sur une demande d'archivage.
BF-26	Gerer les utilisateurs	Administration	Administrateur, Super Admin	Activer un compte et, pour le Super Admin, modifier un role.
BF-27	Consulter un tableau de bord	Tableaux de bord	Tout role	Visualiser les indicateurs pertinents pour son perimetre.
BF-28	Recevoir une notification	Notifications	Tout utilisateur	Etre informe en temps reel d'un evenement le concernant.
BF-29	Utiliser le Copilote IA	Copilote IA	Resp. Logistique & Achat, Admin	Interagir avec l'assistant conversationnel metier.

A ces besoins fonctionnels s'ajoutent des besoins non fonctionnels, identifies a partir des mecanismes techniques effectivement mis en oeuvre dans le code (bibliotheques utilisees, middlewares actifs, structures de journalisation).

Tableau 3.3 — Besoins non fonctionnels identifies

Categorie	Constat effectue dans le code	Element technique
Securite	Authentification par JWT (jeton d'accès + jeton de rafraichissement), mots de passe haches (bcrypt), limitation du taux de requetes sur `/auth/signin` (`signinLimiter`), protection des en-tetes HTTP (`helmet`), anti-enumeration sur les messages d'erreur d'authentification.	jsonwebtoken, bcrypt, express-rate-limit, helmet
Contrôle d'accès	Autorisation a granularite fine combinant role et, ponctuellement, adresse e-mail specifique ; garde d'accès par module (`moduleAccessGuard`).	middlewares `requireRole`, `moduleAccessGuard`, `kpiRoleGuard`
Tracabilite / auditabilite	Journalisation des activites sur plusieurs entites (`ProjectActivity`, `MaintenanceRequestActivity`, `HrRequestActivity`, `CommercialContactStatusHistory`).	tables d'activite dediees (colonne JSON `metadata`)
Reactivite / temps reel	Diffusion immediate des notifications aux utilisateurs connectes via un canal websocket authentifie par JWT.	Socket.IO, rooms `user:<id>`
Portabilite	Interface unique deployee en Flutter Web, consomsee depuis un navigateur, sans distinction de plateforme cliente.	Flutter Web (GoRouter, Dio)
Disponibilite / performance	Information non confirmee par les sources fournies (absence d'elements sur l'infrastructure d'hebergement, la scalabilite ou les objectifs de disponibilite).	A verifier dans le projet

3.5 Diagramme global des cas d'utilisation

Le diagramme ci-dessous presente une vue d'ensemble des cas d'utilisation du systeme PROBAR, organisees en quatre grands domaines fonctionnels (authentification, production industrielle, CRM, demandes internes et administration), conformement aux modules reellement identifies dans le code. Afin de rester lisible, le diagramme regroupe les cas d'utilisation par paquetage plutot que de les représenter a plat.

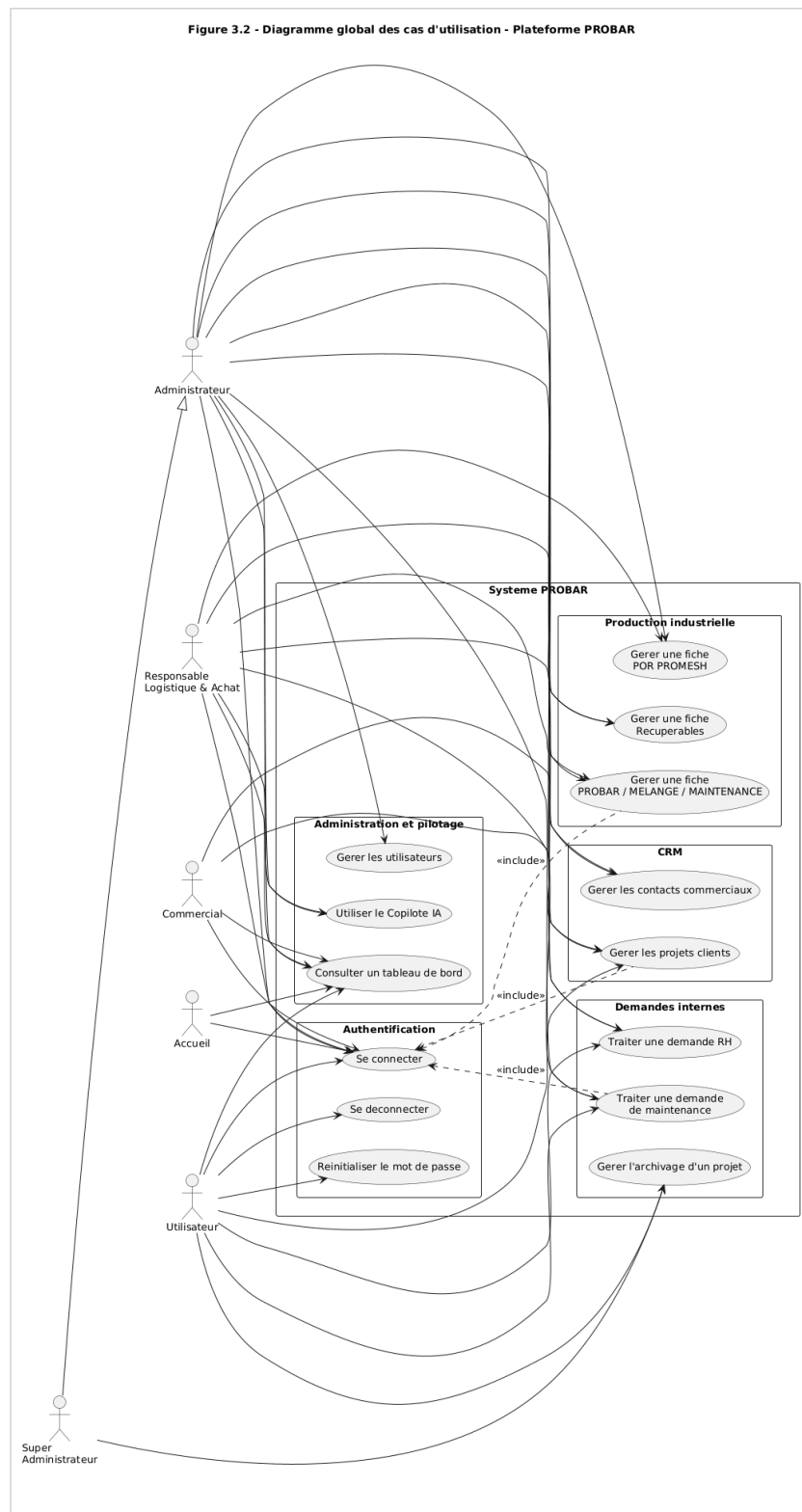


Figure 3.2 — Diagramme global des cas d'utilisation

Les relations d'inclusion ('<<include>>') ne sont utilisées que lorsqu'elles correspondent à une dépendance réellement vérifiée dans le code : la création d'une fiche de production, la gestion d'un projet client et le traitement d'une demande de maintenance nécessitent systématiquement une authentification préalable, reflet direct du middleware 'authRequired' appliqué à chacune de ces routes. Aucune relation d'extension ('<<extend>>') n'a été retenue, faute de scénario optionnel suffisamment distinct dans le code source pour justifier son usage. La relation de généralisation entre Super

Administrateur et Administrateur traduit fidelement la hierarchie de permissions observee dans les middlewares du backend.

La coherence de ce diagramme avec le code a ete verifiee de la maniere suivante : chaque acteur correspond a un role reellement present dans l'enumeration ``User.role`` ou a une combinaison de roles identiques (section 3.3) ; chaque cas d'utilisation correspond a un ensemble de routes API et d'ecrans Flutter effectivement implementes (section 3.4) ; chaque association acteur/cas d'utilisation reflete une regle d'autorisation verifiee dans un middleware du backend (``moduleAccessGuard``, ``requireRole``, ``requireHrReviewer``, ``requireMaintenanceManager``, ``requireRootAdmin``).

3.6 Description detaillee des cas d'utilisation

Les treize cas d'utilisation les plus significatifs du systeme sont decrits ci-dessous selon un formalisme uniforme (acteur principal, acteurs secondaires, objectif, preconditions, declencheur, scenario nominal, scenarios alternatifs, exceptions, postconditions).

UC-01 -- Authentification (Se connecter)

Acteur principal : Tout utilisateur
Acteurs secondaires : Aucun
Objectif : Acceder a l'application avec les droits associes a son role.
Preconditions : Le compte existe et est actif (`isActive = true`).
Declencheur : L'utilisateur soumet le formulaire de connexion (e-mail / mot de passe).

Scenario nominal :

1. L'utilisateur saisit son e-mail et son mot de passe dans l'ecran de connexion.
2. L'application envoie `POST /auth/signin`.
3. Le backend verifie l'existence et l'activation du compte, puis compare le mot de passe (bcrypt).
4. Le backend genere un jeton d'accès JWT (contenant identifiant, e-mail et role) et un jeton de rafraichissement.
5. L'application stocke les jetons et redirige l'utilisateur vers l'interface correspondant a son role.

Scenarios alternatifs :

A1. Mot de passe oublie : l'utilisateur declenche `POST /auth/forgot-password` puis `POST /auth/reset-password`.

Exceptions :

- E1. Compte inexistant ou mot de passe invalide : erreur 401, message generique anti-enumeration.
 E2. Compte inactif (`isActive = false`) : erreur 403.

Postconditions :

Une session authentifiee est etablie ; le jeton de rafraichissement (hache) est enregistre cote serveur.

UC-02 -- Consulter un tableau de bord

Acteur principal : Utilisateur, Commercial, Administrateur, Responsable Logistique & Achat
Acteurs secondaires : Aucun
Objectif : Visualiser les indicateurs pertinents pour son perimetre fonctionnel.
Preconditions : Etre authentifie.
Declencheur : Ouverture de l'ecran tableau de bord.

Scenario nominal :

1. L'utilisateur ouvre l'ecran tableau de bord.
2. L'application determine l'endpoint a appeler selon le role (`/dashboard/user`, `/por-promesh/dashboard`, `/admin-dashboard/stats`).
3. Le service backend agrege les donnees du ou des modules concernes.
4. Le resultat est retourne et affiche sous forme d'indicateurs.

Scenarios alternatifs :

A1. Le Responsable Logistique & Achat consulte uniquement le tableau de bord PROMESH, les endpoints `/dashboard` et `/admin-dashboard` etant hors de son perimetre autorise.

Exceptions :

E1. Role non autorise sur l'endpoint de KPI global : erreur 403 (`kpiRoleGuard`).

Postconditions :

Les indicateurs sont affiches a l'ecran.

UC-03 -- Creer une fiche PROBAR / MELANGE / MAINTENANCE

Acteur principal : Responsable Logistique & Achat

Acteurs secondaires : Administrateur

Objectif : Enregistrer une fiche de production ou d'intervention pour un poste et une machine donnees.

Preconditions : Etre authentifie avec un role appartenant a READ_WRITE_ROLES (admin, superadmin, superadmin2, responsable_logistique_achat).

Declencheur : Ouverture du formulaire de creation de fiche.

Scenario nominal :

1. L'utilisateur selectionne le module concerne (PROBAR, MELANGE ou MAINTENANCE).
2. Il saisit les informations generales (date, machine, poste, operateur en texte libre, heures).
3. Pour le module MELANGE, il complete l'assistant en trois etapes (informations generales, tableau de ravitaillement/consommation, rapport journalier).
4. Il valide le formulaire, ce qui declenche `POST /industrial-records`.
5. Le backend enregistre la fiche (les donnees MELANGE sont stockees dans la colonne JSON `melangeData`) et retourne une confirmation.

Exceptions :

E1. Role non autorise : erreur 403.

E2. Champs obligatoires manquants : erreur 400.

Postconditions :

Une nouvelle ligne est creee dans la table `industrial\_records`.

UC-04 -- Consulter, modifier ou supprimer une fiche PROBAR/MELANGE/MAINTENANCE

Acteur principal : Responsable Logistique & Achat (consultation/modification), Administrateur (suppression)

Acteurs secondaires : Aucun

Objectif : Maintenir a jour ou purger une fiche de production existante.

Preconditions : La fiche existe ; pour la suppression, le role doit appartenir a DELETE_ROLES (admin, superadmin, superadmin2 uniquement).

Declencheur : Selection d'une fiche dans la liste ou l'historique.

Scenario nominal :

1. L'utilisateur selectionne la fiche dans la liste ou l'historique (`GET /industrial-records`).
2. Il consulte le detail (`GET /industrial-records/:id`).
3. Il modifie les champs necessaires et valide (`PUT /industrial-records/:id`).

Scenarios alternatifs :

A1. Suppression : confirmation puis `DELETE /industrial-records/:id`, reservee au role administrateur.

Exceptions :

E1. Fiche introuvable : erreur 404.

E2. Droits insuffisants (modification ou suppression) : erreur 403.

Postconditions :

La fiche est mise a jour ou supprimee en base.

UC-05 -- Creer et completer une fiche POR PROMESH

Acteur principal : Responsable Logistique & Achat

Acteurs secondaires : Aucun

Objectif : Documenter un cycle de production PROMESH complet (personnel, rendement, controle qualite, controle process, arrêts machine, consommations, non-conformites).

Preconditions : Role appartenant a READ_WRITE_ROLES.

Declencheur : Creation ou reprise d'un brouillon de fiche PROMESH.

Scenario nominal :

1. L'utilisateur cree ou reprend un brouillon (`POST /por-promesh/new`).
2. Il parcourt les modules du formulaire (informations generales, personnel, rendement, controle machine, controle qualite, observations, non-conformites).
3. Chaque section est enregistree de maniere incrementale (`PUT /por-promesh/:id`).
4. La fiche recoit un numero de sequence unique, affiche sous la forme « PROMESH-{annee}-{numero} ».

Scenarios alternatifs :

A1. Ajout d'une piece jointe (photo de non-conformite) via `POST /por-promesh/:id/attachments`.

Exceptions :

E1. Fiche deja verrouillee (`isLocked = true`) : la modification est refusee.

Postconditions :

La fiche `por\_promesh` et ses sous-fiches associees sont enregistrees, avec le statut « BROUILLON ».

UC-06 -- Valider (verrouiller) une fiche POR PROMESH

Acteur principal : Responsable Logistique & Achat

Acteurs secondaires : Aucun

Objectif : Cloturer definitivement une fiche apres controle.

Preconditions : La fiche est complete et son statut est « BROUILLON ».

Declencheur : Action « Valider » sur la fiche.

Scenario nominal :

1. L'utilisateur declenche l'action « Valider ».
2. L'application envoie `POST /por-promesh/:id/validate`.
3. Le backend passe le statut a « VALIDE », positionne `isLocked = true` et horodate `validatedAt`.

Exceptions :

E1. Toute tentative de modification posterieure au verrouillage est rejetee par le backend.

Postconditions :

La fiche devient non modifiable.

UC-07 -- Creer et cloturer une fiche Recuperables

Acteur principal : Responsable Logistique & Achat

Acteurs secondaires : Aucun

Objectif : Suivre les chutes et dechets recuperables par machine, ligne et diametre.

Preconditions : La combinaison (module, machine, ligne, poste, date) n'existe pas deja (contrainte d'unicite en base).

Declencheur : Ouverture du formulaire de creation d'une fiche Recuperables.

Scenario nominal :

1. L'utilisateur cree la fiche (statut « en_cours »).
2. Il ajoute une ou plusieurs lignes par diametre (`dechetsKg`, `dechetsProduitFiniKg`).
3. Il cloture la fiche (`PUT /recuperables/:id/terminer`, statut « cloturee »).

Exceptions :

E1. Doublet de fiche pour la meme combinaison module/machine/ligne/poste/date : contrainte d'unicite violee.

Postconditions :

La fiche et ses lignes sont enregistrees ; son statut final est « cloturee ».

UC-08 -- Creer et suivre un projet client

Acteur principal : Utilisateur, Commercial ou Administrateur

Acteurs secondaires : Aucun

Objectif : Piloter un dossier client de sa creation a sa cloture.

Preconditions : Etre authentifie ; pour la modification, etre propriétaire, membre ou administrateur du projet.

Declencheur : Creation d'un nouveau projet ou consultation d'un projet existant.

Scenario nominal :

1. L'utilisateur cree le projet (`POST /projects`).
2. Le projet est associe a une etape du pipeline commercial (`PipelineStage`).
3. Des devis et bons de commande peuvent etre deposes (upload de fichiers).
4. Des actions et rappels sont ajoutees et suivies (`ProjectAction`, `ProjectReminder`).
5. Le projet est suivi via la vue Kanban (`GET /pipeline/kanban`) et son fil d'activite (`ProjectActivity`).

Scenarios alternatifs :

A1. Export Excel de la liste des projets (`GET /projects/export`).

Exceptions :

E1. Acces refuse si l'utilisateur n'est ni propriétaire, ni membre, ni administrateur (politique `requireOwnerOrAdmin`).

Postconditions :

Le projet est cree ou mis a jour, avec son historique d'activite trace.

UC-09 -- Gerer un contact commercial et planifier une relance

Acteur principal : Commercial

Acteurs secondaires : Administrateur

Objectif : Suivre la prospection commerciale et relancer un client potentiel.

Preconditions : Etre authentifie avec un role eligible.

Declencheur : Creation d'un nouveau contact ou consultation d'un contact existant.

Scenario nominal :

1. L'utilisateur cree le contact commercial (`POST /commercial-contacts`).
2. Il ajoute une action commerciale associee.
3. Il planifie une relance (`POST /commercial-contacts/:id/relances`).
4. La relance est notifiee et, le cas echeant, synchronisee au calendrier Google.

Exceptions :

E1. Echec de synchronisation Google Calendar : trace dans `googleCalendarError`, sans bloquer l'enregistrement de la relance.

Postconditions :

Le contact, ses actions et ses relances sont enregistres ; l'historique de statut est trace (`CommercialContactStatusHistory`).

UC-10 -- Soumettre et traiter une demande de maintenance

Acteur principal : Utilisateur (soumission)

Acteurs secondaires : Responsable Logistique & Achat ou Administrateur (traitement)

Objectif : Signaler une panne d'equipement et en assurer le suivi jusqu'a resolution.

Preconditions : Etre authentifie.

Declencheur : Constat d'une panne par un utilisateur.

Scenario nominal :

1. L'utilisateur cree le ticket avec photos eventuelles (`POST /maintenance-requests`, statut « en_attente »).
2. Une notification est creee en base et diffusee en temps reel au gestionnaire de maintenance.
3. Le gestionnaire accepte le ticket (`PUT /:id/accept`).
4. Il affecte un technicien (`PUT /:id/assign`), qui demarre (`PUT /:id/start`) puis cloture (`PUT /:id/complete`) l'intervention.

Scenarios alternatifs :

A1. Rejet motive du ticket (`PUT /:id/reject`).

Exceptions :

E1. Role non habilite a traiter le ticket : erreur 403 (`requireMaintenanceManager`).

Postconditions :

Le ticket suit le cycle de statuts « en_attente -> acceptee -> en_cours -> terminee » (ou « refusee »).

UC-11 -- Soumettre et traiter une demande RH

Acteur principal : Utilisateur (soumission)
Acteurs secondaires : Administrateur (traitement)
Objectif : Dematerialiser une demande de conge ou de sortie.
Preconditions : Etre authentifie.
Declencheur : Besoin de conge ou de sortie exceptionnelle.

Scenario nominal :

1. L'utilisateur cree la demande avec justificatifs eventuels (`POST /hr-requests`).
2. Un courriel de notification est genere.
3. Le gestionnaire RH accepte ou rejette la demande (`PUT /:id/accept` ou `/reject`).
4. La demande est marquee comme traitee (`PUT /:id/process`).

Scenarios alternatifs :

A1. Export PDF de la demande (`GET /:id/pdf`).

Exceptions :

E1. Role non habilite : erreur 403 (`requireHrReviewer`).

Postconditions :

La demande suit le cycle de statuts « en_attente -> acceptee/refusee -> traitee ».

UC-12 -- Gerer les utilisateurs

Acteur principal : Administrateur (activation)
Acteurs secondaires : Super Administrateur (changement de role)
Objectif : Administrer les comptes de la plateforme.
Preconditions : Role admin, superadmin ou superadmin2.
Declencheur : Consultation de la liste des utilisateurs.

Scenario nominal :

1. L'administrateur consulte la liste des utilisateurs (`GET /admin/users`).
2. Il active ou desactive un compte (`PUT /admin/users/:id/active`).
3. Le super administrateur modifie eventuellement le role d'un utilisateur (`PUT /admin/users/:id/role`).

Exceptions :

E1. Tentative de changement de role par un simple administrateur : erreur 403 (reserve a superadmin/superadmin2).

Postconditions :

Le statut d'activation ou le role de l'utilisateur cible est mis a jour.

UC-13 -- Demander et approuver l'archivage d'un projet

Acteur principal : Utilisateur, Commercial ou Administrateur (demande)

Acteurs secondaires : Administrateur racine (approbation)

Objectif : Encadrer la mise en archive ou le desarchivage d'un dossier projet.

Preconditions : Le projet existe ; le demandeur en est propriétaire ou administrateur.

Declencheur : Decision d'archiver ou de desarchiver un projet.

Scenario nominal :

1. L'utilisateur cree la demande (`POST /archive-requests`, statut « pending »).
2. Des messages peuvent etre echanges (`POST /archive-requests/:id/messages`).
3. Le compte administrateur racine approuve ou rejette la demande (`PUT /:id/approve` ou `/reject`).
4. En cas d'approbation, le projet est effectivement archive ou desarchive.

Exceptions :

E1. Seule l'adresse e-mail administrateur racine peut approuver ou rejeter, independamment du role porte par le compte connecte.

Postconditions :

La demande passe au statut « approved » ou « rejected ».

3.7 Diagrammes d'activite

Conformement aux processus reellement identifies dans le code, huit diagrammes d'activite ont ete produits, couvrant l'authentification, la creation de fiches de production (cas generique et cas particulier du module MELANGE), la consultation de l'historique, la modification et la suppression d'une fiche, le depot d'un document et le traitement d'une demande interne.

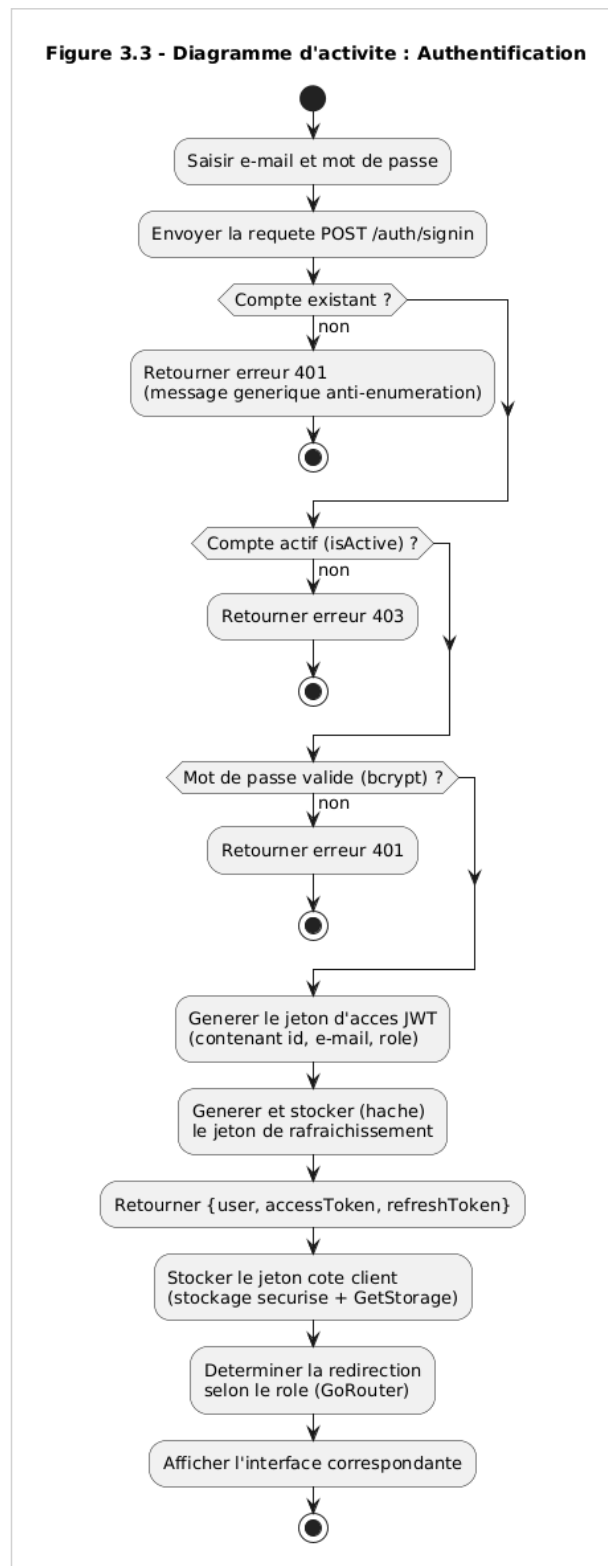


Figure 3.3 — Diagramme d'activite : Authentification

Ce diagramme retranscrit fidelement le flux reel de `POST /auth/signin` : verification de l'existence du compte, verification de son activation, verification du mot de passe (bcrypt), puis emission du jeton JWT et redirection selon le role. L'infrastructure d'authentification multifacteur (MFA), bien que presente dans le code (`services/mfa.service.js`, routes `/auth/mfa/verify` et `/auth/mfa/resend`), n'est pas invoquee par le flux de connexion actuel et n'a donc pas ete representee dans le scenario nominal (voir « Points a verifier »).

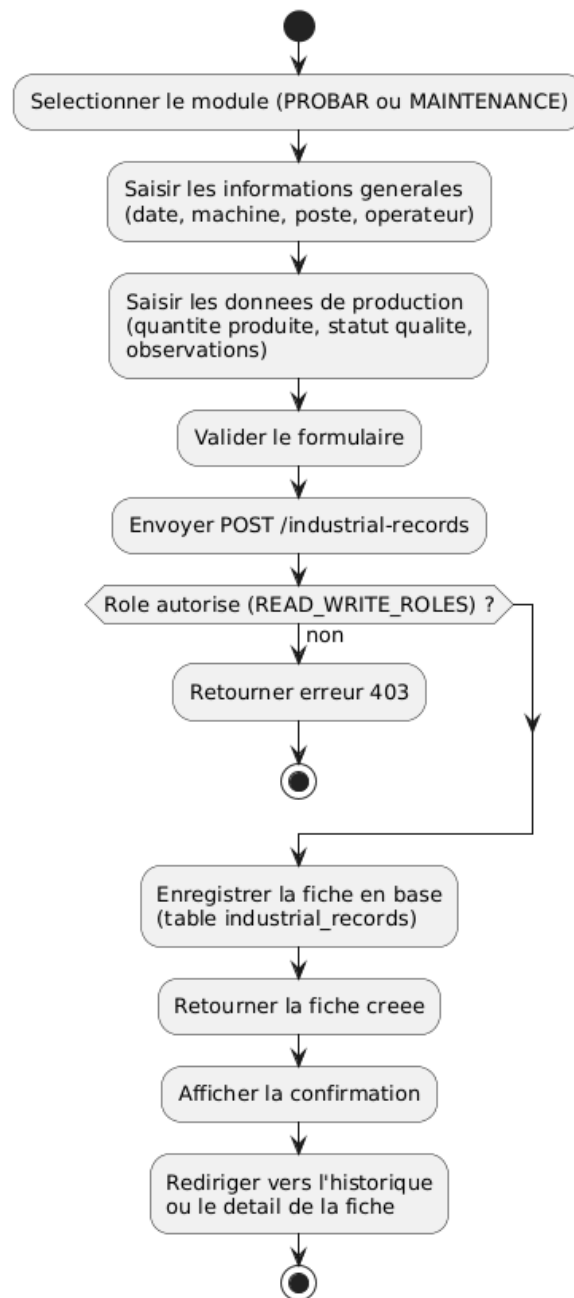
Figure 3.4 - Diagramme d'activite : Creation d'une fiche de production (PROBAR / MAINTENANCE)

Figure 3.4 — Diagramme d'activite : Creation d'une fiche de production (PROBAR / MAINTENANCE)

Ce diagramme illustre le parcours generique de creation d'une fiche pour les modules PROBAR et MAINTENANCE, controle par `READ\_WRITE\_ROLES` cote backend.

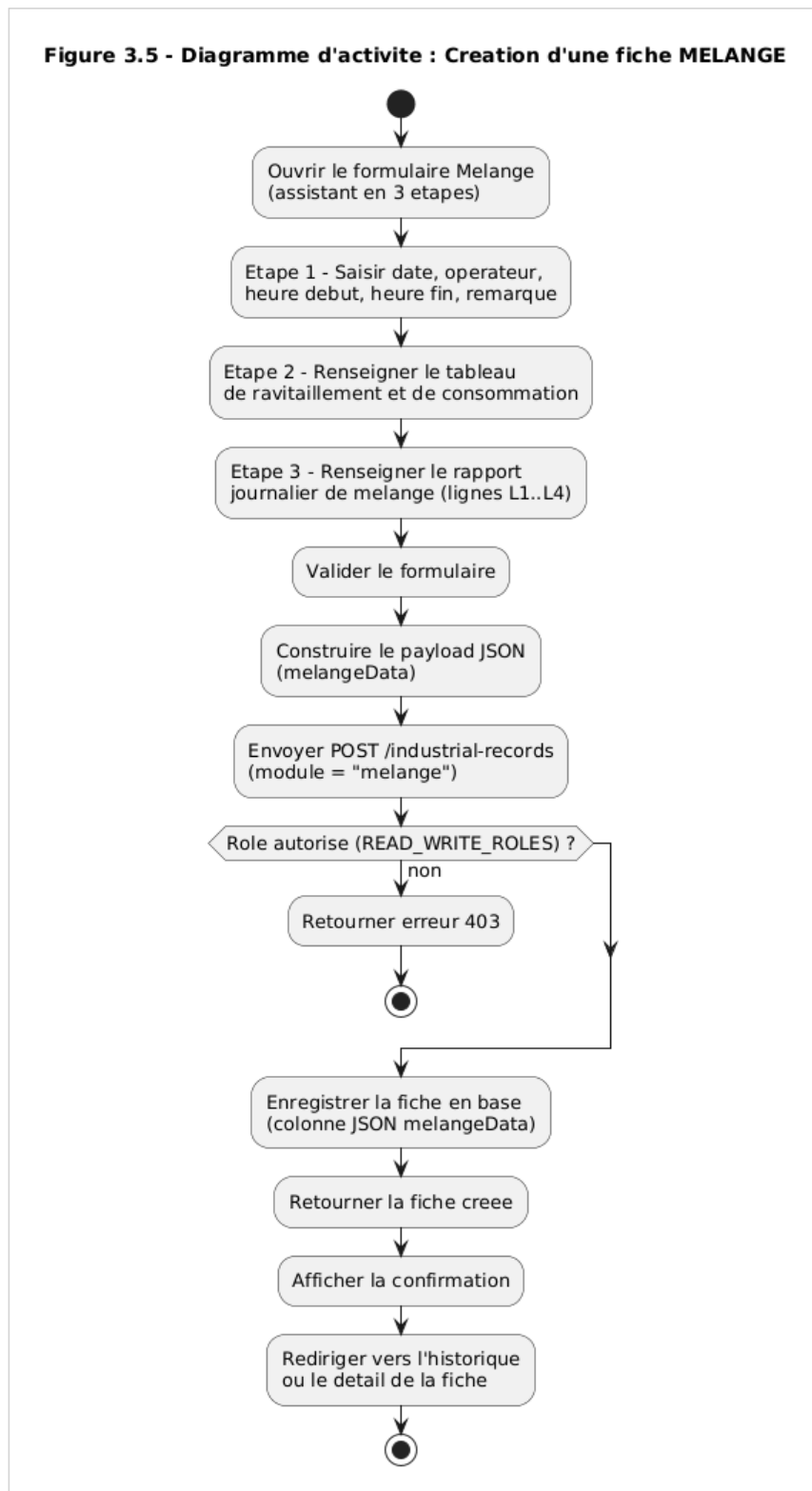
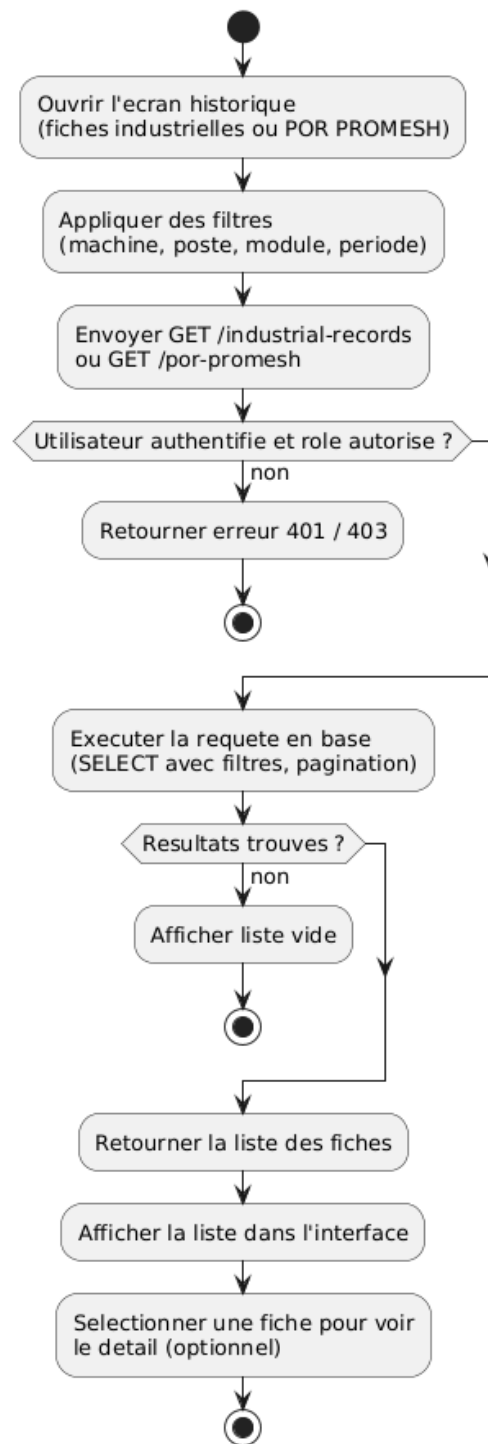
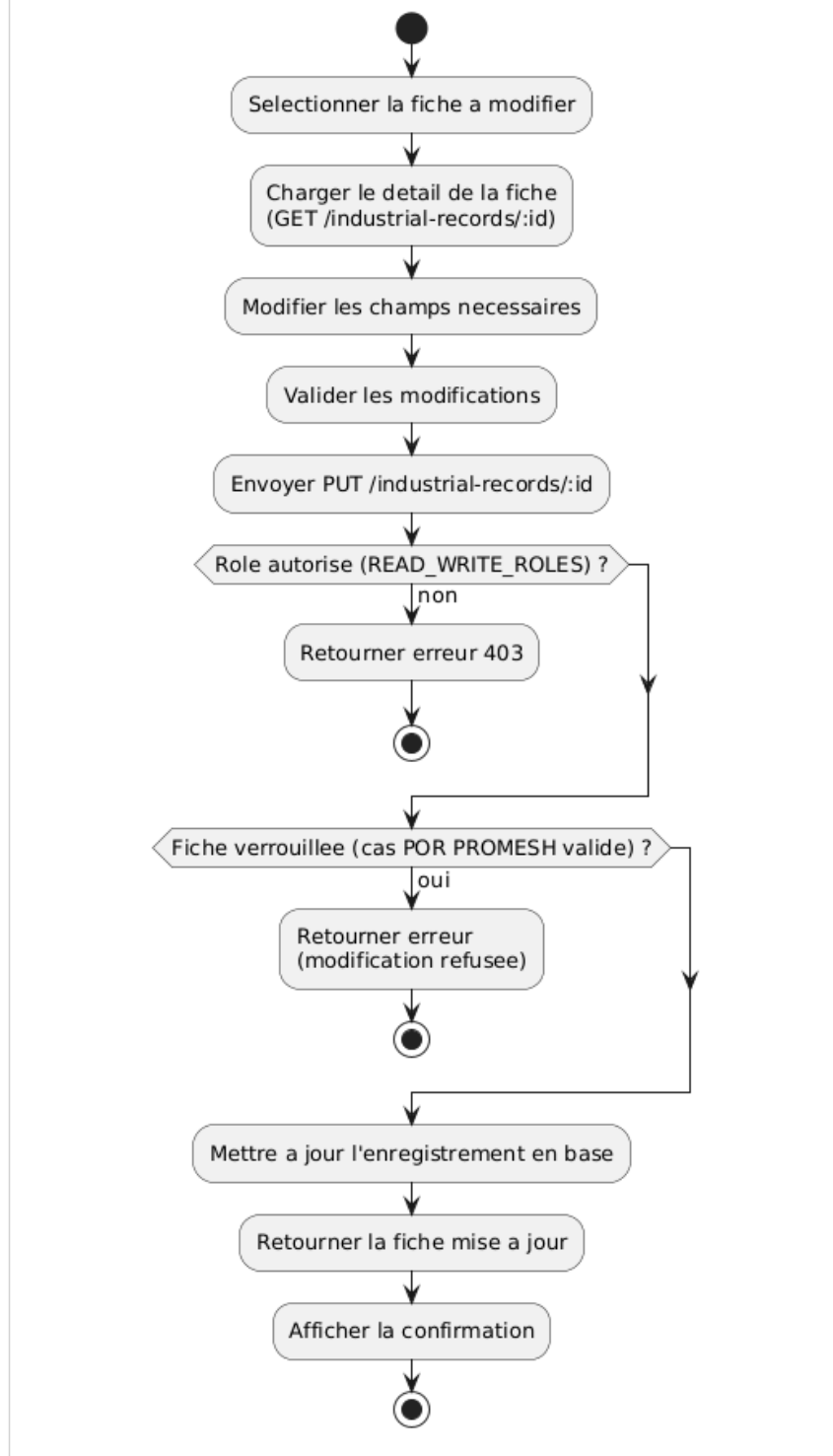
Figure 3.5 - Diagramme d'activite : Creation d'une fiche MELANGE

Figure 3.5 — Diagramme d'activite : Creation d'une fiche MELANGE

Ce diagramme illustre l'assistant de saisie en trois etapes propre au module MELANGE, dont les donnees sont regroupees dans la colonne JSON `melangeData` de la table `industrial\_records`.

Figure 3.6 - Diagramme d'activite : Consultation de l'historique*Figure 3.6 — Diagramme d'activite : Consultation de l'historique*

Ce diagramme decrit la consultation filtree de l'historique des fiches (industrielles ou POR PROMESH), avec verification systematique du role avant l'execution de la requete.

Figure 3.7 - Diagramme d'activite : Modification d'une fiche*Figure 3.7 — Diagramme d'activite : Modification d'une fiche*

Ce diagramme met en evidence le cas particulier des fiches POR PROMESH verrouillees apres validation, pour lesquelles toute modification est refusee par le backend.

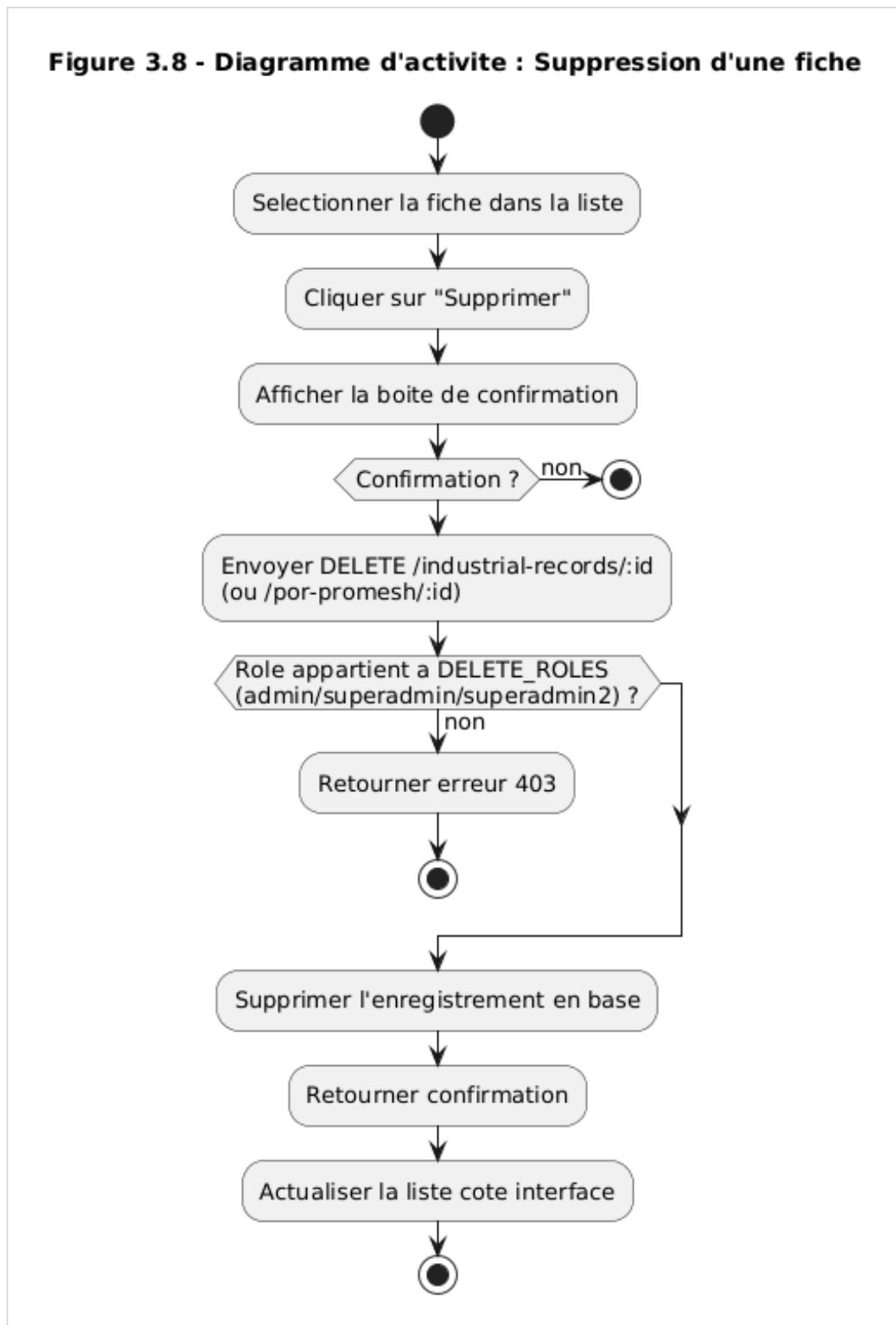
Figure 3.8 - Diagramme d'activite : Suppression d'une fiche

Figure 3.8 — Diagramme d'activite : Suppression d'une fiche

Ce diagramme illustre la restriction de la suppression aux seuls roles administrateurs ('DELETE_ROLES'), plus stricte que celle appliquee a la lecture et a la modification.

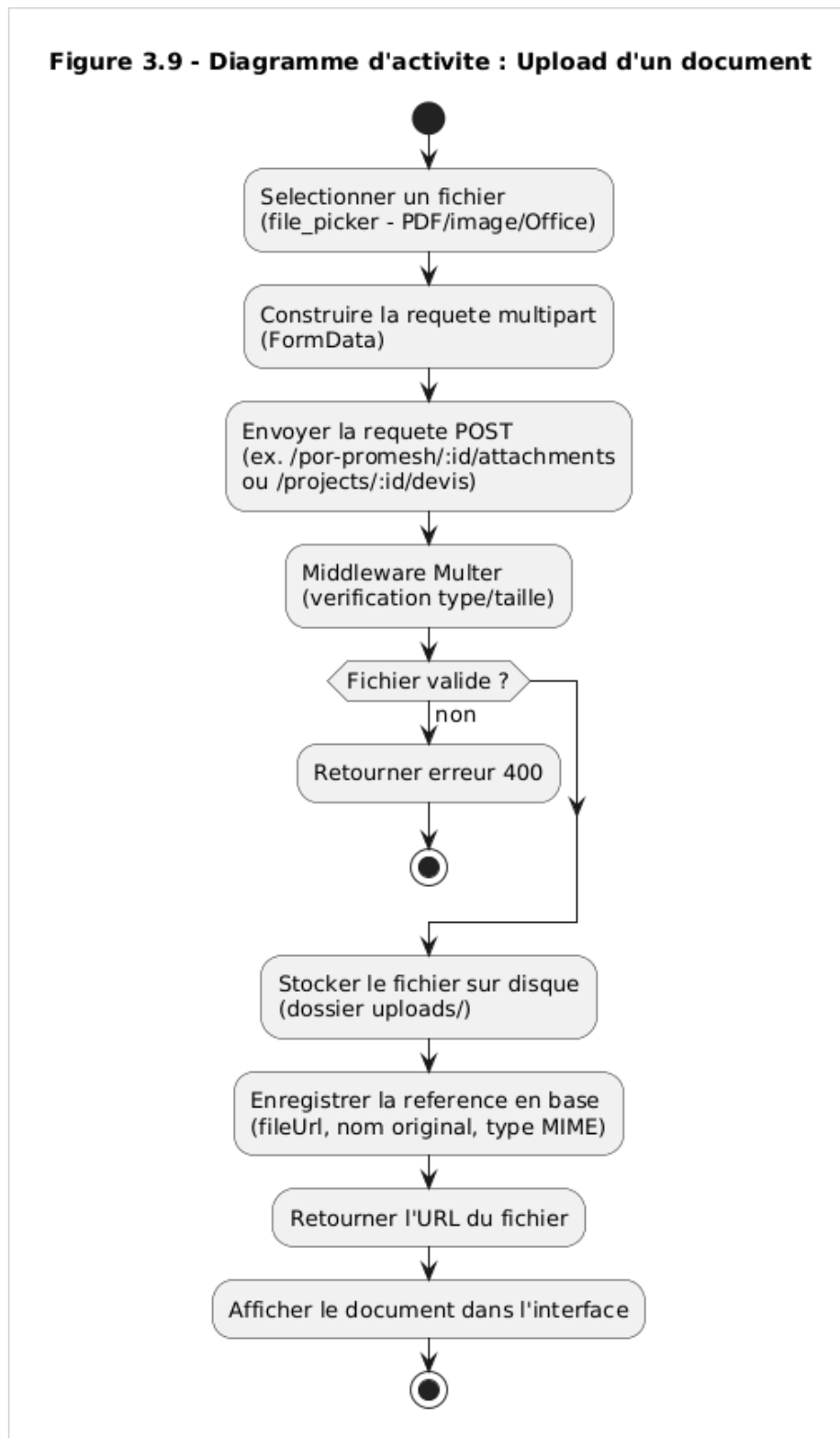
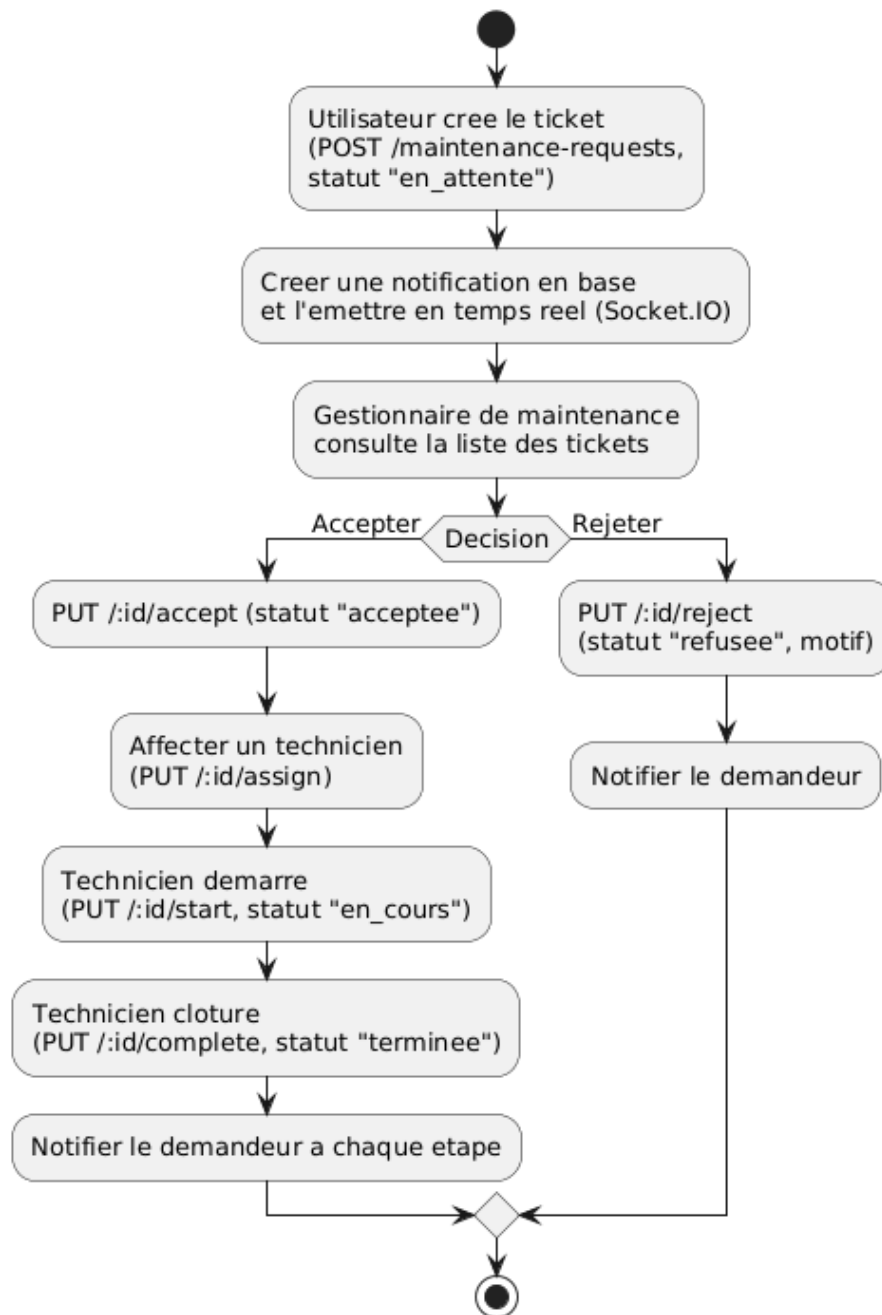
Figure 3.9 - Diagramme d'activite : Upload d'un document

Figure 3.9 — Diagramme d'activite : Upload d'un document

Ce diagramme decrit le depot d'un fichier (piece jointe, devis, bon de commande) via le middleware Multer. Aucune etape de traitement OCR n'existe dans le processus reel : le fichier est stocke et lie tel quel.

Figure 3.10 - Diagramme d'activite : Traitement d'une demande de maintenance*Figure 3.10 — Diagramme d'activite : Traitement d'une demande de maintenance*

Ce diagramme illustre le cycle de vie complet d'un ticket de maintenance, de sa creation a sa cloture, avec notification du demandeur a chaque changement d'etat.

3.8 Diagramme de classes

L'analyse des 48 modeles Sequelize presents dans `src/models/` et de leurs associations declarees dans `src/models/associations.js` a permis de construire le diagramme de classes du systeme. Afin de rester lisible, ce diagramme est presente en une vue globale simplifiee, centree sur le coeur du modele metier, puis en trois vues detaillees par domaine fonctionnel : Production, CRM et Demandes internes.

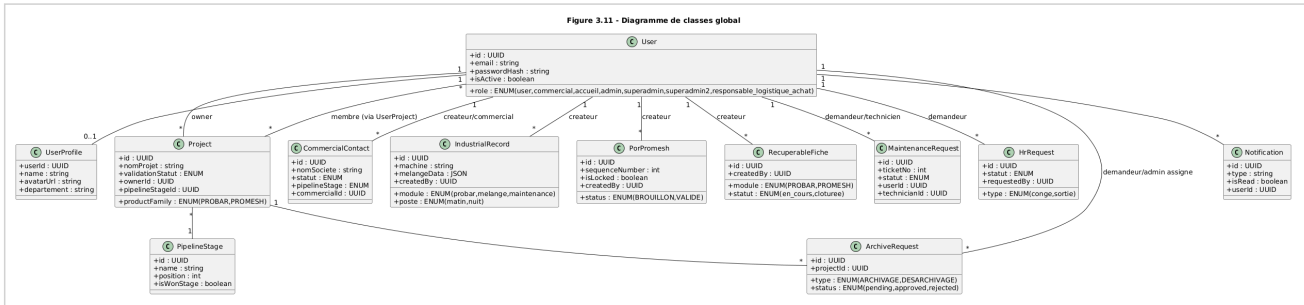


Figure 3.11 — Diagramme de classes global

La classe `User` constitue le pivot du modele : elle est reliee par des relations un-a-un ou un-a-plusieurs a la quasi-totalite des entites metier (createur d'une fiche, proprietaire d'un projet, demandeur d'un ticket, destinataire d'une notification), refletant le fait que chaque enregistrement du systeme est systematiquement rattache a l'utilisateur qui en est a l'origine.

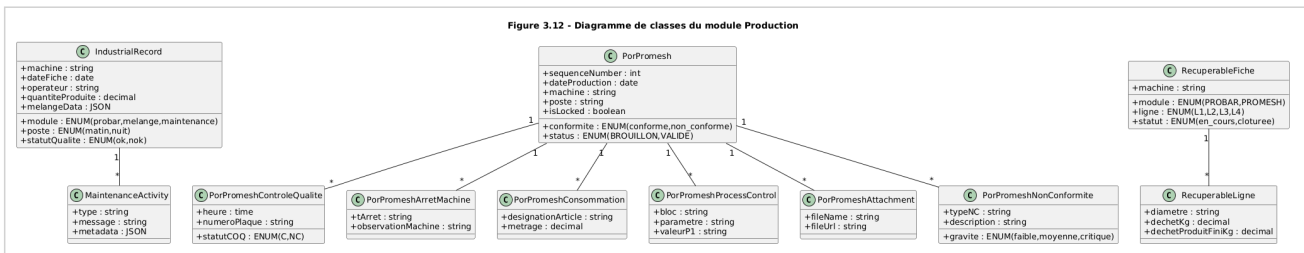


Figure 3.12 — Diagramme de classes du module Production

Le module Production met en evidence deux structures de donnees distinctes : `IndustrialRecord`, table unique partagee par les fiches PROBAR, MELANGE et MAINTENANCE (discriminees par le champ `module`), et `PorPromesh`, dont la structure est nettement plus riche puisqu'elle est composee de six sous-fiches associees (contrôle qualité, arrêts machine, consommations, contrôle process, pieces jointes et non-conformites), toutes reliees par une cle étrangere en cascade.

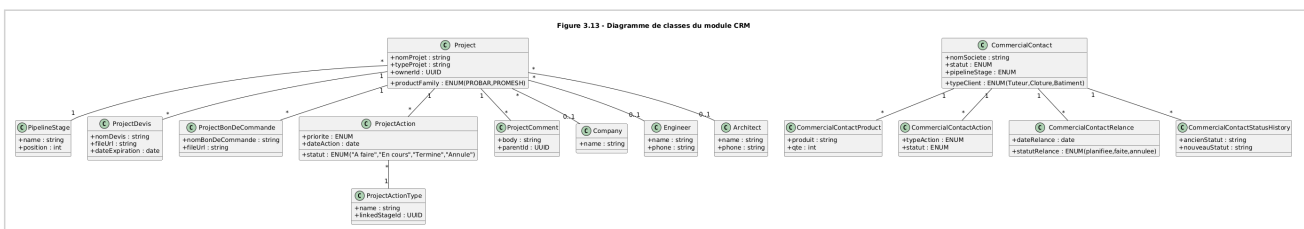


Figure 3.13 — Diagramme de classes du module CRM

Le module CRM articule deux axes : le suivi de projet (`Project`, associe a une etape de pipeline, a des devis, des bons de commande, des actions et des commentaires) et la gestion des contacts commerciaux (`CommercialContact`, associe a des produits, des actions, des relances et un historique de changement de statut).

Figure 3.14 - Diagramme de classes du module Demandes internes

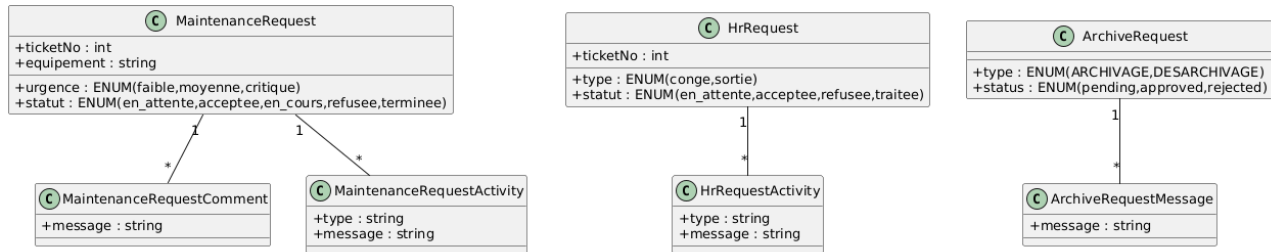


Figure 3.14 — Diagramme de classes du module Demandes internes

Les trois workflows de demandes internes (maintenance, ressources humaines, archivage) partagent une structure similaire : une entite principale porteuse d'un statut, associee a une ou plusieurs entites de suivi (commentaires, messages, activites), sans toutefois partager de classe commune dans le code — il s'agit de trois implementations paralleles, non d'une hierarchie de classes unifiee.

Tableau 3.4 — Classes principales du modele metier

Classe	Table	Rôle métier
User	users	Compte utilisateur, porteur du rôle d'autorisation.
UserProfile	user_profiles	Informations de profil complémentaires (nom, avatar, service RH).
Project	projects	Dossier client / chantier suivi par le CRM.
PipelineStage	pipeline_stages	Étape du cycle de vente auquel un projet est rattaché.
CommercialContact	commercial_contacts	Prospect ou contact commercial.
IndustrialRecord	industrial_records	Fiche de production PROBAR, MELANGE ou MAINTENANCE (discriminée par le champ 'module').
PorPromesh	por_promesh	Fiche de production PROMESH, avec six sous-fiches associées.
RecuperableFiche	recuperable_fiches	Suivi des chutes et déchets récupérables par ligne de production.
MaintenanceRequest	maintenance_requests	Ticket de panne équipement (workflow de demande interne).
HrRequest	hr_requests	Demande de congé ou de sortie.
ArchiveRequest	archive_requests	Demande d'archivage ou de désarchivage d'un projet.
Notification	notifications	Notification utilisateur, doublée d'un événement temps réel Socket.IO.

3.9 Diagrammes de sequence

Six diagrammes de sequence ont été produits pour les scénarios les plus représentatifs du système, en détaillant systématiquement le trajet réel de la requête : interface Flutter, service Flutter (client Dio), route Express, middleware d'autorisation, contrôleur, service métier, ORM Sequelize et base de données PostgreSQL. Les codes d'erreur HTTP représentés (401, 403, 404, 400) correspondent aux réponses effectivement émises par les routes concernées, et les fragments `alt`/`opt` ne sont utilisés que lorsqu'ils correspondent à une branche conditionnelle réellement présente dans le code du contrôleur ou du service.

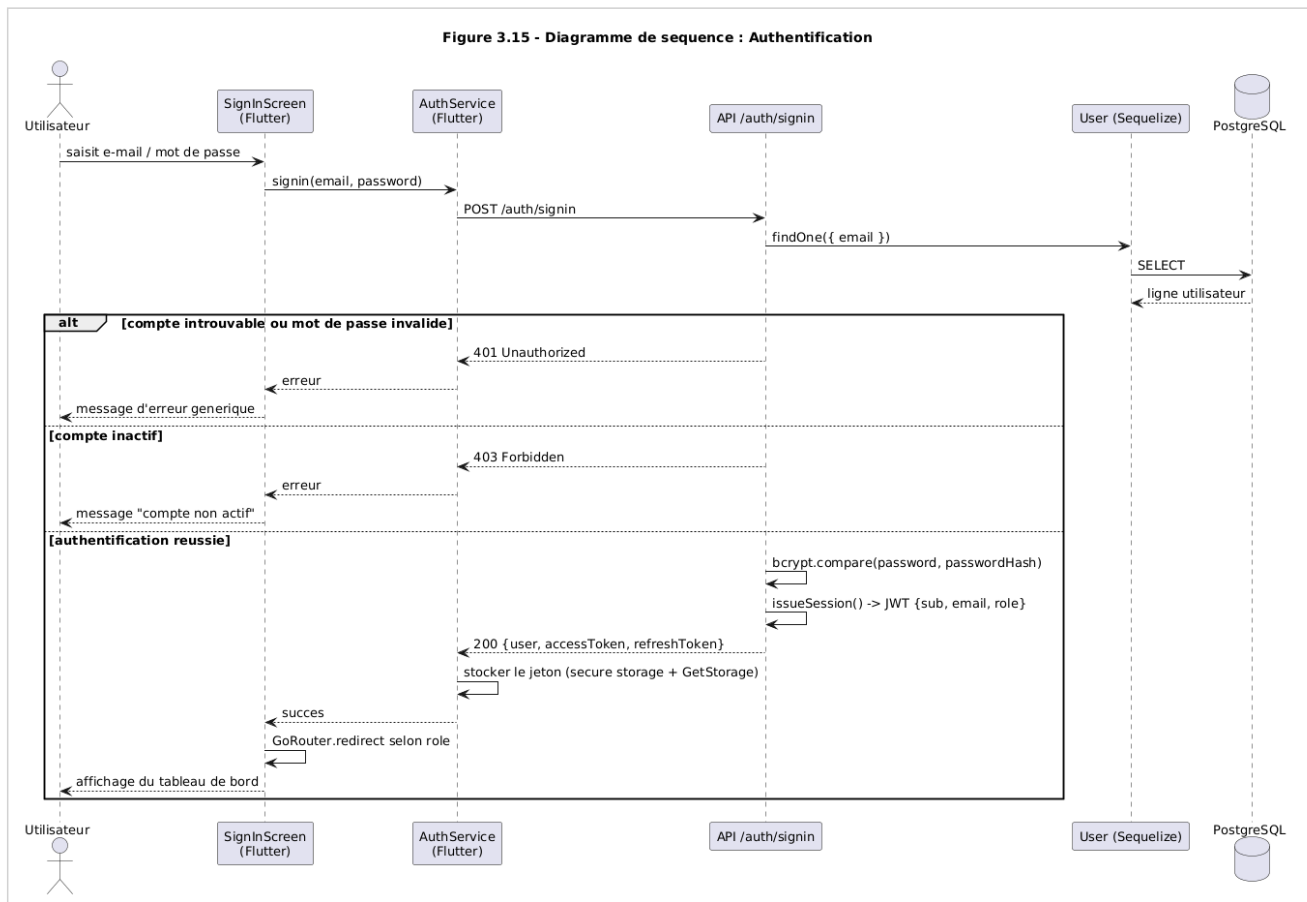


Figure 3.15 — Diagramme de sequence : Authentification

Ce diagramme détaille les trois issues possibles de la tentative de connexion (echec d'identification, compte inactif, succes), a l'aide d'un fragment `alt` correspondant exactement aux branches conditionnelles du contrôleur `/auth/signin`.

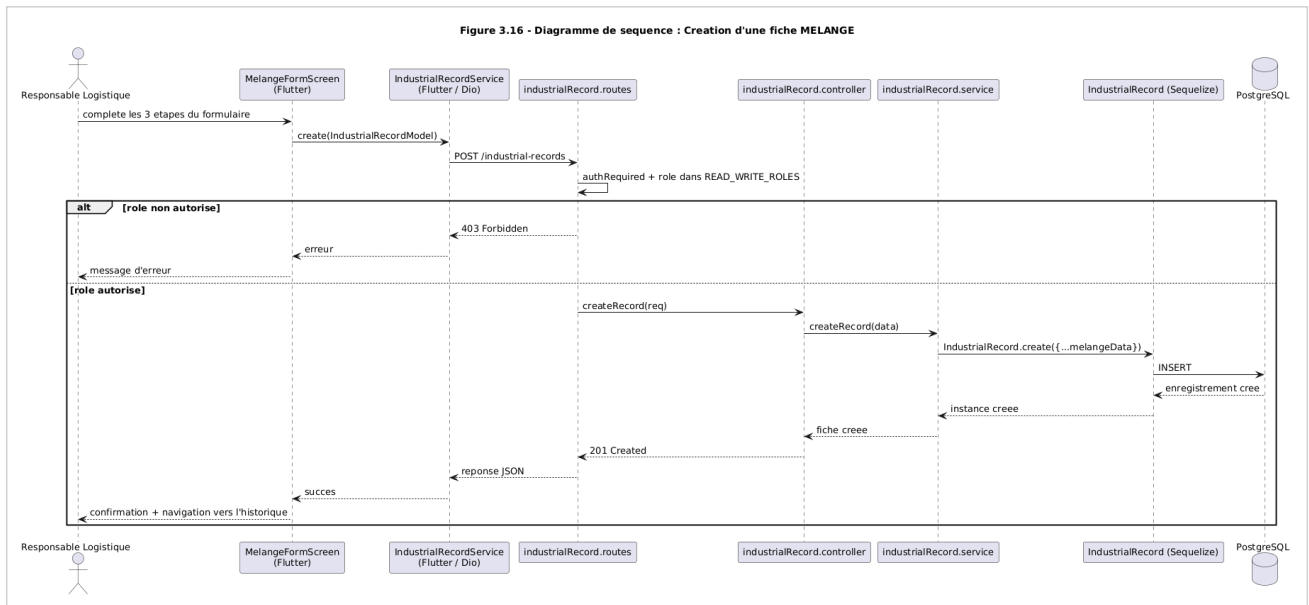


Figure 3.16 — Diagramme de sequence : Creation d'une fiche MELANGE

Ce diagramme trace le trajet complet de la requete depuis l'ecran Flutter jusqu'a l'enregistrement en base, en passant par la verification du role ('READ_WRITE_ROLES').

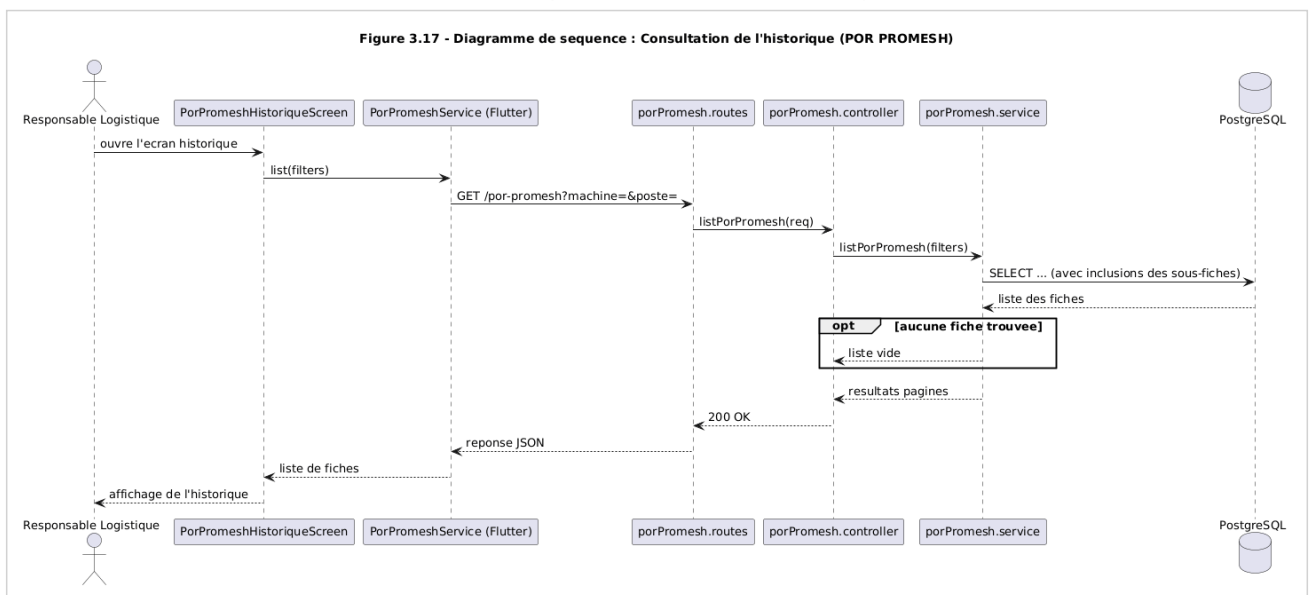


Figure 3.17 — Diagramme de sequence : Consultation de l'historique

Ce diagramme illustre la consultation paginee et filteree des fiches POR PROMESH, avec un fragment 'opt' pour le cas d'une liste vide.

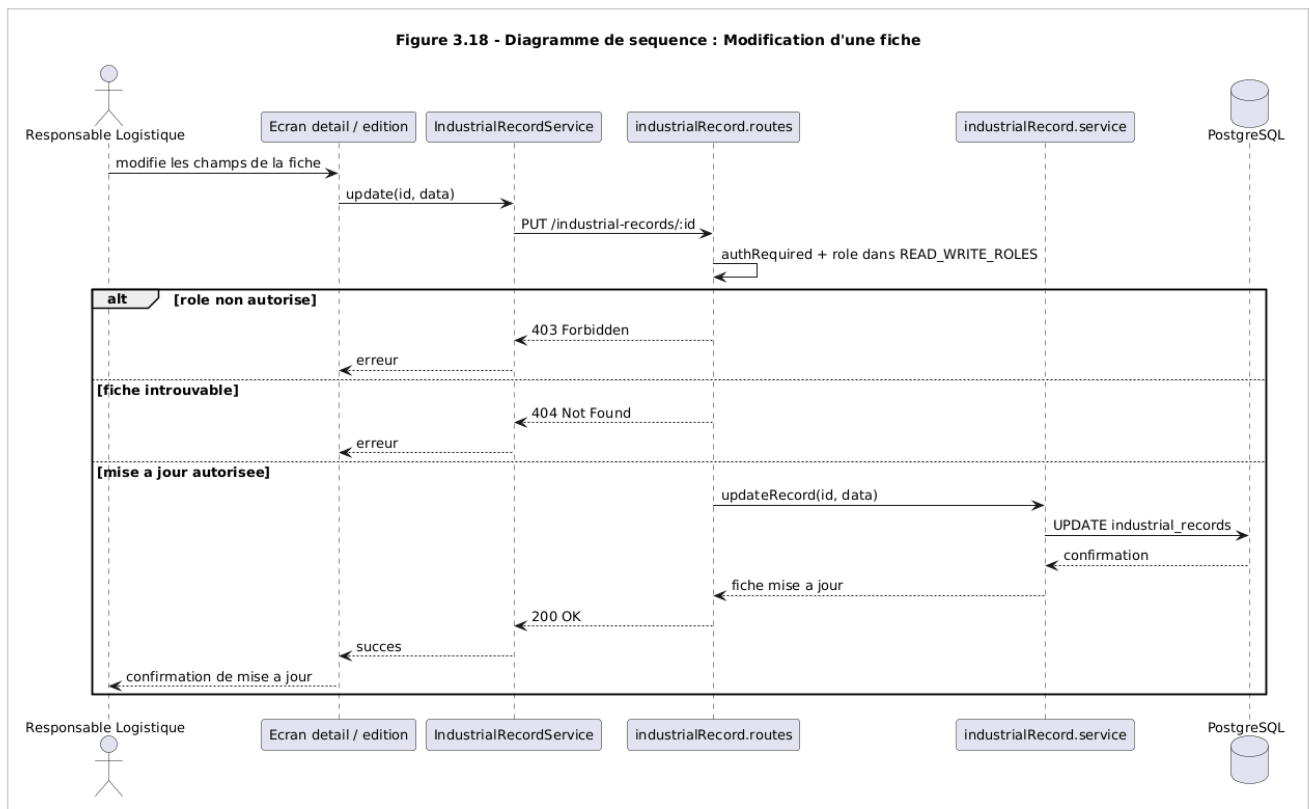


Figure 3.18 — Diagramme de sequence : Modification d'une fiche

Ce diagramme presente les trois issues possibles de la requete `PUT /industrial-records/:id` (role non autorise, fiche introuvable, mise a jour reussie), a l'aide d'un fragment `alt`.

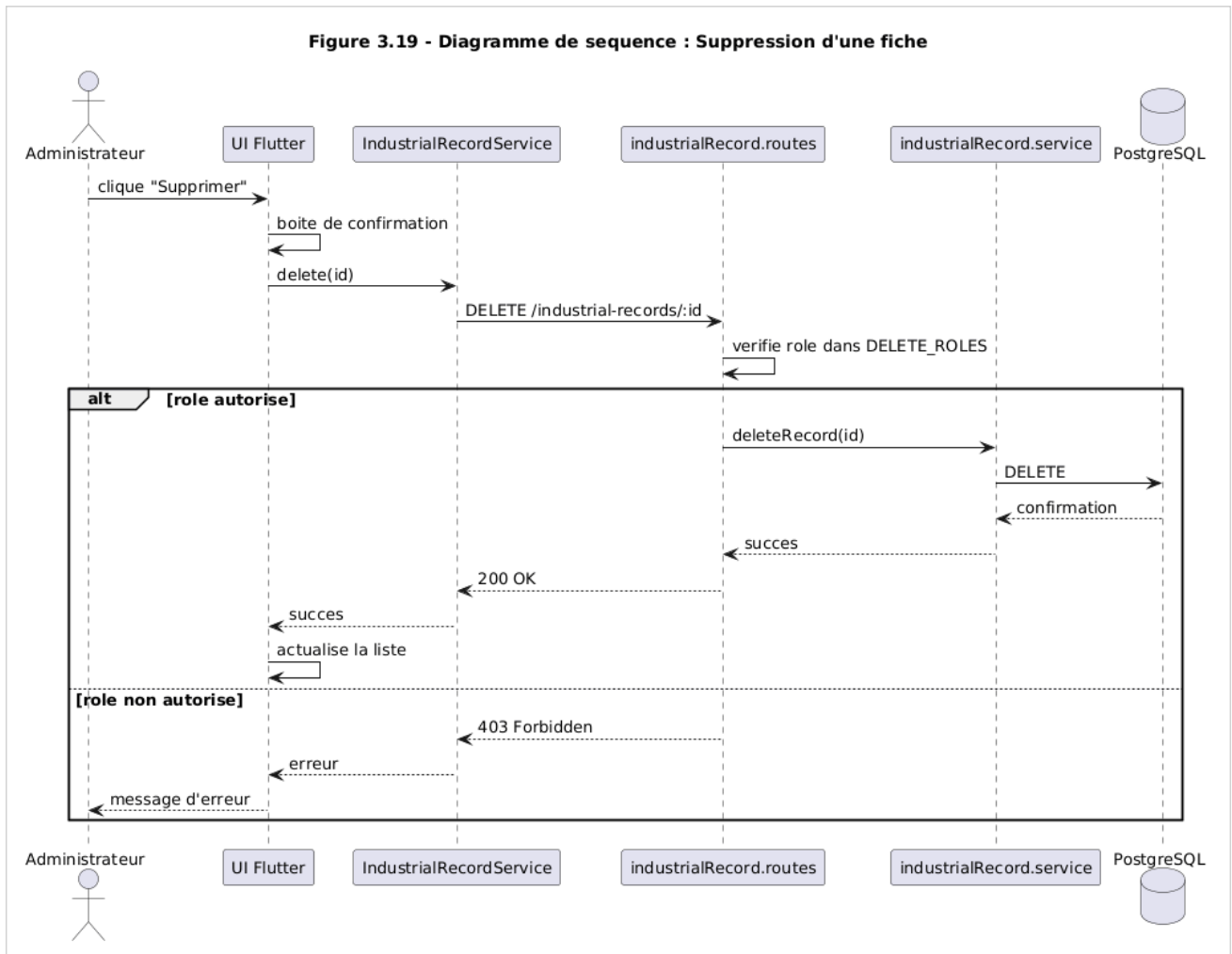


Figure 3.19 — Diagramme de sequence : Suppression d'une fiche

Ce diagramme met en evidence la verification du role au sein de 'DELETE_ROLES', plus stricte que celle appliquee a la lecture et a la modification ('READ_WRITE_ROLES').

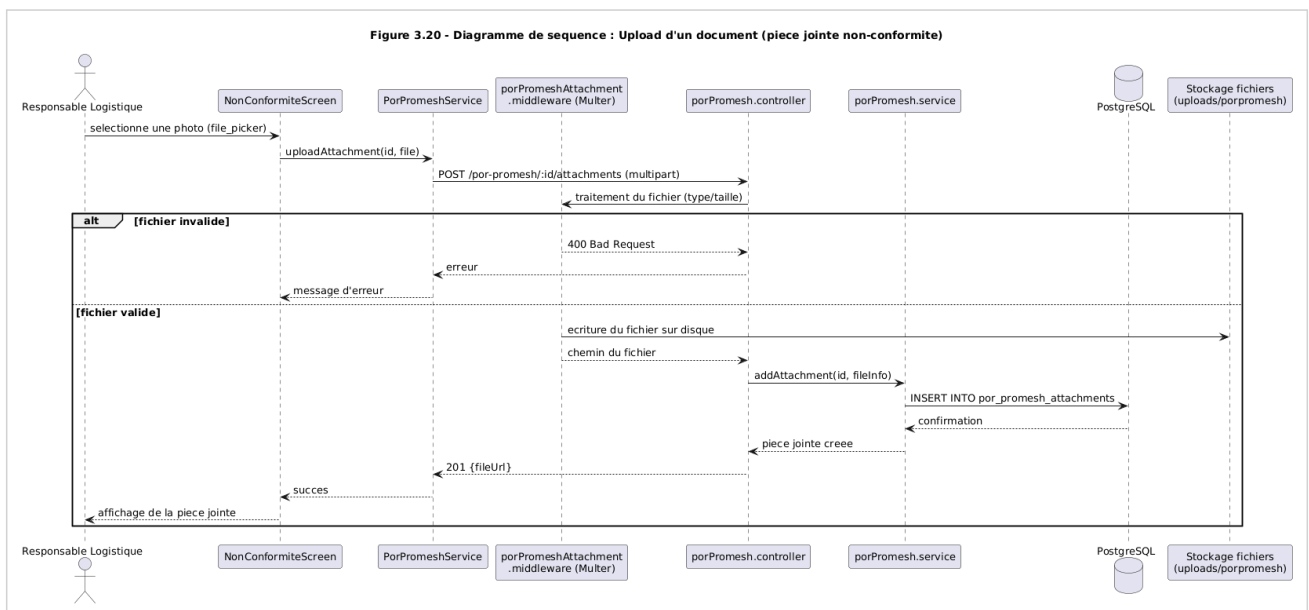


Figure 3.20 — Diagramme de sequence : Upload d'un document

Ce diagramme detaille le depot d'une piece jointe de non-conformite PROMESH, en integrant la

verification du fichier par le middleware Multer avant ecriture sur le systeme de fichiers.

3.10 Architecture fonctionnelle du systeme

L'architecture fonctionnelle du systeme PROBAR est organisee en couches successives, conformément au decoupage effectivement observe entre le frontend Flutter Web et le backend Express/Sequelize/PostgreSQL.

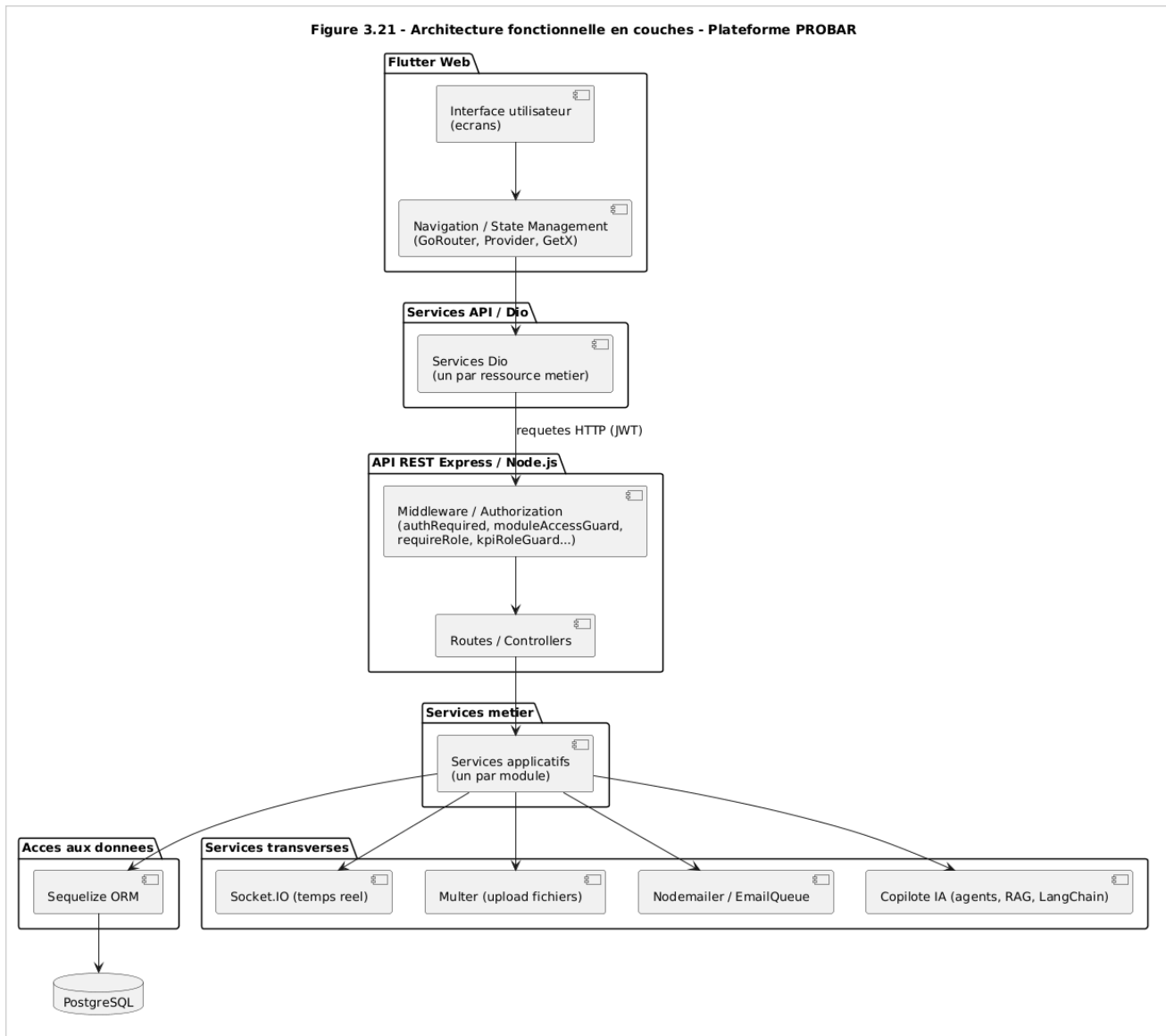


Figure 3.21 — Architecture fonctionnelle en couches

Tableau 3.5 — Role de chaque couche de l'architecture

Couche	Role
Flutter Web	Presente les ecrans et gere la navigation conditionnee au role ('GoRouter.redirect') ainsi que l'etat applicatif (combinaison de 'Provider' et 'GetX' observee dans le code).
Services API / Dio	Encapsulent les appels HTTP vers chaque ressource backend ; injectent automatiquement le jeton JWT dans l'en-tete 'Authorization'.
Middleware / Authorization	Chaine de securite executee avant tout traitement metier : authentication ('authRequired'), garde d'accès par module ('moduleAccessGuard'), garde de role ('requireRole', 'kpiRoleGuard', 'requireHrReviewer', 'requireMaintenanceManager', 'requireRootAdmin').
Routes / Controllers	Recoivent la requete HTTP et delegue immediatement au service metier correspondant.
Services metier	Portent la logique applicative propre a chaque module (regles de gestion, creation de notifications, verrouillage de fiche, etc.).

Couche	Role
Sequelize ORM / PostgreSQL	Couche de persistance, avec migrations versionnees (`src/migrations/`).
Services transverses	Socket.IO pour la diffusion temps reel, Multer pour le stockage de fichiers, Nodemailer/EmailQueue pour les courriels, et un sous-systeme d'intelligence artificielle (agents, RAG, outils MCP, LangChain) consomme par le module Copilote.

3.11 Synthese de la conception

La conception presentee dans ce chapitre traduit des choix architecturaux coherents, verifies a chaque etape par confrontation directe avec le code source. Trois enseignements principaux se degagent de cette analyse.

Premierement, le modele de roles retenu (sept valeurs d'enumeration, complete par des restrictions ponctuelles fondees sur l'adresse e-mail) est plus fin que ne le laisserait supposer une lecture superficielle du systeme : le role ``responsable_logistique_achat`` concentre a lui seul l'essentiel des responsabilites operationnelles de production, de maintenance et de logistique, tandis que les roles ``admin``/``superadmin``/``superadmin2`` assurent une supervision transversale. Cette concentration se retrouve directement dans le middleware ``moduleAccessGuard``, qui constitue le point unique de verite pour l'acces aux modules industriels.

Deuxiemement, la separation entre le module de production generique (``IndustrialRecord``, table unique partagee par les fiches PROBAR, MELANGE et MAINTENANCE) et le module de production specialise (``PorPromesh``, structure en six sous-fiches) refletent deux niveaux de maturite fonctionnelle distincts au sein du meme domaine metier : le premier privilegie la flexibilite d'un champ JSON libre (``melangeData``), tandis que le second impose un modele relationnel strict, mieux adapte a la tracabilite fine exigee par un processus de controle qualite normalise.

Troisiemement, l'architecture en couches (Flutter Web, services API, middleware d'autorisation, controleurs, services metier, ORM, base de donnees), completee par des services transverses reels (Socket.IO, stockage de fichiers, messagerie, copilote IA), demontre une separation des responsabilites conforme aux bonnes pratiques d'ingenierie logicielle, meme si certains elements — en particulier le mecanisme d'autorisation hybride role/e-mail et l'absence d'activation du MFA malgre son implementation complete — constituent des points d'amelioration a documenter dans la suite du rapport.

Cette synthese constitue le socle sur lequel s'appuiera le chapitre suivant, consacre a la realisation technique du systeme.

3.12 Conclusion

L'analyse du code source de la plateforme PROBAR a permis d'identifier un systeme sensiblement plus riche — et different dans sa terminologie — que ne le laissait supposer le cahier des charges initial : il ne s'agit pas d'un progiciel de gestion generique dote de modules Finance/Facturation/Expedition/OCR, mais d'une plateforme hybride combinant un CRM complet (projets, contacts commerciaux, pipeline commercial) et un suivi de production industrielle structure autour de deux familles de produits reelles, PROBAR et PROMESH, complete par des workflows de demandes internes (maintenance, ressources humaines, archivage) et un copilote conversationnel fonde sur l'intelligence artificielle.

Cette conception, restituee fidelement par les diagrammes de cas d'utilisation, de classes, d'activite et de sequence presentes dans ce chapitre, ainsi que par l'architecture fonctionnelle en couches, constitue une base solide et verifiee pour aborder, dans le chapitre suivant, la realisation technique du systeme.

* Points a verifier dans le projet

Le tableau ci-dessous recense, de maniere transparente, les elements qui n'ont pas pu etre confirmes avec certitude a partir des sources explorees (code source du backend et du frontend) et qui doivent etre valides aupres de l'equipe projet avant integration definitive dans le rapport.

Tableau 3.6 -- Points a verifier dans le projet

Element concerne	Information trouvee	Probleme eventuel	Recommandation
Modules Finance / Factures / Expéditions / Matières premières	Aucune route, aucun modele et aucun ecran correspondant a ces modules n'a ete trouve dans le code explore.	Ces modules, mentionnes dans le cahier des charges initial, semblent ne pas exister dans le systeme reel ; le module le plus proche est « Recuperables » (suivi des chutes/dechets), de portee fonctionnelle differente.	Confirmer aupres de l'equipe projet si ces modules existent dans une autre base de code, sont planifies, ou doivent etre retires du cahier des charges du rapport.
Fonctionnalite OCR / scan de documents	Aucune integration OCR (Tesseract, Google/Azure/AWS Vision, etc.) n'a ete trouvee ; les depots de documents sont de simples televersements de fichiers sans extraction de contenu.	La fonctionnalite « scan de document » evoquee dans le cahier des charges ne correspond a aucune implementation reelle.	Clarifier si une fonctionnalite OCR est prevue dans une iteration future ou si le terme « scan » designait en realite le televersement simple de photographies.
Roles finance / production / maintenance / operateur	L'enumeration reelle du role utilisateur ne contient pas ces valeurs ; le role `responsable_logistique_achat` couvre une large part de ce perimetre.	Risque de confusion terminologique entre les roles organisationnels de l'entreprise et les roles techniques du systeme.	Verifier aupres de l'equipe si ces appellations correspondent, dans l'organisation reelle, au role unique `responsable_logistique_achat`.
Permissions precises du role Accueil	Aucune restriction ni elargissement specifique n'a ete identifie dans le code au-dela de son exclusion de la liste des commerciaux eligibles aux relances.	Le perimetre fonctionnel exact de ce role reste incertain.	Interroger l'equipe projet sur l'usage reel de ce role en production.
Restrictions par adresse e-mail codee en dur	Confirmees presentes dans le code pour l'administrateur racine, le gestionnaire RH, le gestionnaire de maintenance et certains comptes « admin restreint ».	Ce mecanisme constitue potentiellement une dette technique (couplage fort entre une regle metier et une valeur en configuration) plutot qu'un choix de conception definitif.	Verifier aupres de l'equipe de developpement si ce mecanisme est amene a evoluer vers un systeme de permissions plus generique avant d'etre presente comme un choix assume dans le rapport.
Activation du MFA (authentification multifacteur)	L'infrastructure complete existe (modeles, service, routes) mais n'est pas invoquee par le flux `/auth/signin` actuel.	Ecart entre une fonctionnalite implementee et son activation effective en production.	Confirmer si une activation est prevue ou si cette fonctionnalite est en pause temporaire.
Fournisseur du modele de langage (LLM) du Copilote IA	Plusieurs documents internes suggerent une evolution du fournisseur (passage d'un fournisseur a un autre).	Le fournisseur effectivement actif en configuration n'a pas ete verifie dans le cadre de cette analyse.	Verifier la configuration d'environnement active avant de nommer le fournisseur dans le rapport final.
Role exact du champ « operateur »	Il s'agit d'un champ texte libre saisi par l'utilisateur connecte (generalement le Responsable Logistique & Achat), non d'un compte utilisateur distinct.	Un ecart peut exister entre la pratique organisationnelle (plusieurs operateurs de terrain) et la modelisation informatique (un seul compte partage saisissant plusieurs noms).	Confirmer aupres de l'equipe si cette modelisation correspond a l'usage reel sur le terrain.

Element concerne	Information trouvee	Probleme eventuel	Recommandation
Perimetre fonctionnel du role Accueil sur les projets CRM	Certains ecrans frontend suggerent un usage (tableau de statistiques projets pour l'accueil), sans restriction backend specifique confirmee.	Le perimetre precis de ce role sur le module CRM n'est pas totalement determine.	Verifier avec l'equipe si des regles d'accès complementaires existent en dehors du code explore.
Besoins non fonctionnels de disponibilite et de performance	Aucune information sur l'infrastructure d'hebergement, la scalabilite ou les objectifs de disponibilite n'a ete trouvee dans les sources explorees.	Ces exigences, si elles existent, ne sont pas documentees dans le code.	Recueillir ces informations aupres de l'equipe d'exploitation ou d'infrastructure si elles doivent figurer dans le rapport.